* DNS (Domain Name System) 용어
- Recursive/Cache 네임서버/DNS서버 : 관리하는/위임받은 도메인 없이 사용자 호스트로부터 질의가 들어오면 자신의 캐시에 저장된 정보 또는 반복적 질의를 통해 그 결과를 사용자 호스트에 응답해주는 네임 서버 -> 일반적으로 ISP 업체가 제공해주는 DNS 서버가 해당함
- Authoritative 네임서버/DNS서버 : 관리하는/위임받은 도메인을 가지고 있는 네임 서버 -> 특정 도메인에 대한 정보를 관리하면서 해당 도메인에 대한 질의에만 응답해주는 서버
- 존(Zone) : 네임서버가 관리하는 도메인 영역
- 존 파일(zone file) : 관리도메인에 대한 정보를 담고 있는 파일
- 재귀적 질의 (Recursive Query) : 사용자 호스트가 Recursive 네임서버로 질의할 때 사용되는 방식으로 Recursive 네임서버로 대상 도메인의 리소스 레코드 정보를 조회해서 응답해다랄는 질의를 의미한다. Recursive 네임 서버는 자신의 캐시 데이터를 조회한 후 데이터가 있으면 사용자 호스트로 그 결과를 반환하고, 데이터가 없으면 최상위 root 네임서버로부터 질의 대상 도메인 네임서버까지 반복적 질의를 수행한 후 그 결과를 사용자 호스트에 반환한다.
- 반복적 질의 (Iterative Query) : Recursive 네임서버가 도메인을 관리하는 각 네임서버로 질의할 때 사용하는 방식, 최상위 Root 네임서버로부터 계층구조에 따라 대상 도메인 네임서버까지 각 네임서버가 응답하는 순차적으로 반복하여 진행함

* 윈도우 호스트의 DNS 질의 동작 방식 : 검색순서는 다음과 같다. 로컬 DNS 캐시 -> hosts.ics -> hosts 파일 -> 없으면 시스템에 설정된 네임 서버로 요청

cf. hosts.ics 파일 : 윈도우에서 인터넷 연결 공유 (ICS) 사용시 클라이언트의 IP를 강제로 지정하는 기능을 가짐.

* 리눅스/유닉스 기준 네임서버 관련 설정 파일
- /etc/resolv.conf : 시스템 기본 네임서버 설정 정보를 담고 있음
- /etc/hosts : 도메인/호스트명과 IP주소 매핑정보를 담고 있는 파일, 네임서버 질의전 먼저 참조 되는 파일, 파밍 공격에 사용되는 사례 자주보고됨
- /etc/host.conf : DNS 질의 순서를 지정하는 파일, order 에 명시된 순서로 질의를 수행함

* 네임서버 질의/응답 패킷 : 응답데이터가 512 바이트 이하일 경우 (일반적인 경우) UDP/53 사용, 512 바이트 초과일 경우 TCP/53 사용

* DNS Cache 관련 명령어
- ipconfig / displaydns (로컬 DNS 캐시 정보 조회) -> TTL 해당 캐시 정보를 유지하는 시간을 의미하며 TTL이 만료되면 해당 정보는 삭제 된다, DNS 스푸핑을 당할 경우 캐시에 조작된 주소 정보가 저장 됨
- ipconfig / flushdns (로컬 DNS 캐시 정보 삭제하기)

* DNS Lookup 명령어 : forward DNS loockup(순방향 룩업) / reverse DNS lookup(역방향 룩업)이 있음
- 순방향 룩업 : 도메인명을 통해서 IP주소를 알아내는 질의, 역방향 룩업 : IP주소를 통해서 도메인 명을 알아내는 질의
- nslookup / dig 명령을 통해 DNS lookup 확인 가능
- 질의 유형 : Zone file에 저장된 각각의 레코드 타입을 의미함. 질의 유형을 지정하지 않으면 A 레코드에 대한 질의를 수행함

A(Address -> 순방향 lookup), ANY(도메인에 대한 모든 레코드 질의, DRDos 공격에 악용 됨), MX(Mail Exchanger), NS(Name Server), SOA(Start Of Authority), TXT(Text -> 도메인에 대한 텍스트 정보 질의를 의미하며 대표적으로 SPF 레코드 정보가 있음, DRDos 공격에 악용됨), PTR(Pointer-> IP에 대한 도메인 정보 질의, A유형 반대), AXFR(Authoriative Zone Transfer->존 버전에 상관없이 무조건 존 전송 요청), IXFR(Incremental Zone Transfer->존 버전을 비교하여 상위 버전일 경우 존 전송 요청)

* nslookup 명령어 : 명령 실행 후 대화형 모드로 질의 / nslookup [도메인] [네임서버] -> 질의할 도메인을 지정한 네임서버로 질의함

* dig 명령어 : nslookup을 대체하기 위한 목적으로 만들어진 명령어로 Linux 및 Unix 계열의 OS는 기본적으로 설치되어 있음. -> dig [@네임서버] 도메인 [쿼리유형] [쿼리옵션]

-> 쿼리옵션 : +norecurse (authoritative 네임서버에 반복적 질의를 수행하여 정상 응답 여부를 점검), +tcp(53/tcp 허용 여부를 점검함), +trace(계층적 위임 상태 점검, 최상위 루트 도메인부터 최종 질의대상 도메인까지 계층구조에 따른 질의 수행)

* whois 명령어 : 해당 도메인의 등록정보, 네트워크 할당정보를 조회하기 위한 명령어 -> 해당 도메인명이 다른 사람에 의해 사용중인지 점검하는 용도로 많이 사용함
- whois 도메인명 : 해당 도메인의 등록정보/소유정보, 네임서버 정보 등을 조회함
- whois IP 주소 : 해당 주소의 네트워크 대역을 관리하는 대행자 정보, 네트워크 할당 정보 등을 확인할 수 있다

* DNS 스푸핑 공격 : 희생자에게 전달되는 DNS 응답을 조작하거나 DNS 서버의 캐시정보를 조작하여 희생자가 의도하지 않은 주소로 접속하게 만드는 공격 -> 희생자 입장에서는 정상적인 URL로 접속하지만 실제로는 공격자가 만든 가짜 사이트로 접속하게 됨 -> 스니핑을 이용한 DNS 스푸핑 / DNS 캐시 포이즈닝으로 나뉨

* DNS 스푸핑은 DNS 질의/응답에 사용하는 UDP 프로토콜의 비연결 특성 취약점을 이용한 공격 -> 클라이언트는 DNS 서버와 연결 상태를 유지하지 않기 때문에 DNS 질의/응답을 식별하기 위한 Transaction ID, 출발지/목적지 주소만 일치하면 먼저 수신한 응답을 신뢰하고 이후에 수신한 응답을 모두 폐기하는 특성을 이용한 공격임

* 스니핑 기반의 DNS Spoofing 공격 : 희생자가 DNS 질의를 수행하면 공격자가 이를 스니핑하고 있다가 정상 응답보다 빠르게 희생자에게 조작된 웹사이트 IP 정보를 담은 DNS 응답을 보내 정상 주소를 입력해도 조작된 주소로 접속하게 만드는 공격 기법 , 조작된 응답 이후에 도착하는 정상 응답은 수신한 응답을 신뢰하는 특성으로 인해 폐기 됨 -> 일반적으로 스니핑을 통한 DNS 스푸핑 공격은 동일 네트워크 내에서 발생하기 때문에 상대적으로 멀리 떨어져 있는 DNS 서버 보다 공격자가 빠르게 조작된 응답을 보낼 수 있음

=> 대응책 : 스니핑이 기본적으로 이용되기 때문에 스니핑 자체를 탐지하고 차단해야함, 중요한 사이트의 IP주소에 대해서는 DNS 질의 보다 우선 순위가 높은 hosts 파일에 등록해서 관리하도록 한다

* DNS Cache Poisoning 공격 : DNS 서버의 캐시 정보를 조작하는 공격을 DNS Cache Posioning 공격이라고 함 -> DNS 서버 자체를 공격하기 때문에 DNS 서버의 캐시 정보가 일정시간 유지되는 동안 해당 서버에 접근하는 다수의 사용자가 조작된 DNS 응답을 수신하여 대규모의 보안 사고가 발생할 수 있다.

- 공격자는 공격 대상 DNS 서버(Recursive DNS)에 조작할 도메인 질의를 다수 보냄 -> 공격자는 공격 대상 DNS 서버가 반복적 질의를 수행하는 동안에 다수의 조작된 DNS 응답을 보냄. 공격자가 다수의 응답을 보내는 이유는 공격 대상 DNS 서버가 반복적 질의 시 사용하는 Transaction ID 와 출발지 Port를 모르기 때문에 랜덤하게 생성해서 일단 보내는 것임 (스니핑이 가능할 경우 Transaction ID 와 port 를 알아 낼 수 있음) -> 공격자의 조작된 응답 중 정상 응답보다 먼저 일치하는 응답이 있으면 조작된 주소 정보가 공격 대상 DNS 서버의 캐시에 저장되고, 이를 질의하는 사용자는 조작된 주소의 사이트로 접속하게 됨

cf. 생일 공격 : 공격자가 랜덤한 Transcation ID를 다수 생성해서 DNS Cache Poisoning 공격을 하는 것을 생일 공격 기법을 이용했다고 말함. 생일 공격은 우리가 예상하는 확률보다 실제 확률이 뜻밖에 높다는 이론에 기반함

=> 대응책 : 네임서버의 소프트웨어를 최신 버전 상태로 유지하여 알려진 취약점에 의한 공격이 발생하지 않도록 함, 도메인 관리용 DNS 서버는 재귀적 질의를 허용하지 않도록 설정하고 제한된 사용자가 사용하는 Recursive DNS 서버라면 해당 사용자로 제한해서 허용하도록 한다, DNSSEC(DNS Security Extensions) 기술을 활용함

cf. 데이터 위변조 공격에 취약한 DNS 문제점을 근본적으로 보안 개선하기 위해 IETF에 의해 2005년에 완성된 국제표준 기술, 기존의 DNS에 공개키 암호화 방식의 보안 기능을 추가 부여하여 DNS의 보안성을 대폭 강화하는 역할을 함

* DNS 서버 보안 = Bind DNS 보안
- 도메인 네임서버는 도메인의 존 데이터 관리 측면에서 원본 존 데이터를 관리하는 마스터 네임서버와 이를 동기화해서 사용하는 슬레이브 네임서버로 구분함 -> 존 데이터는 관리하는 도메인 영역 정보로 다양한 레코드 유형에 따른 데이터를 가지고 있으며 일반적으로 관리자가 설정한 존 파일을 읽어 들여 존 데이터를 구성함 / 마스터에 있는 원본 존 데이터를 슬레이브가 동기화 하는 작업을 '존 전송' 이라고 표현함. TCP/53 사용

-> 존 전송은 마스터에 있는 원본 존 데이터를 슬레이브가 동기화하고 신뢰성이 필요하기 때문에 TCP/53 사용 !!
- 마스터와 슬레이브 네임서버는 서로 동일한 기능을 수행하며 부하분산을 통해 안전성을 높이는 역할을 함
- 리졸버(Resolver) : 네임서버로 질의를 수행하여 그 결과를 응용 프로그램에 반환해주는 소프트웨어 모듈/라이브러리를 의미

* 존 파일 이해하기 -> named.conf 에 존설정을함 (네임서버 설정 파일)
cf. 리버스 도메인 : reverse lookup을 위한 특수한 도메인 (IP 에 대한 도메인명 질의를 위함)
cf. 리버스 존 파일 : 리버스 도메인에 대한 존 파일을 의미함

-> options 의 directory 항목이 존 파일이 위치한 경로를 나타냄
-> 존 파일은 네임서버 설정 시 가장 중요한 도메인 데이터베이스 파일로 네임서버 기동시에 존 파일을 읽어 들여 존 데이터를 구성함

host_name [TTL] class record_type data 로 구성됨

예시) www 300 IN A 192.168.56.100 또는 ns1 IN A 192.168.56.10

-> host_name : 등록할 호스트명 지정, TTL : 레코드의 TTL 값 지정 하는데 미지정시 default 값을 적용 받음, class : 인터넷 클래스를 의미하는 IN 지정, record_type : 레코드 유형을 지정함, data : 레코드 유형에 따른 데이터 설정

cf. 리버스 존 파일 설정을 살펴보면 PTR 레코드 유형을 이용하여 IP주소에 대한 도메인명을 설정하는것을 볼 수 있음

* 존 파일의 주요 리소스 레코드 유형 및 역할
- A(Address) : 호스트 IP 정보
- AAAA : 호스트 IP 정보
- MX(Mail Exchanger) : 도메인의 메일서버 정보
- NS(Name Server) : 도메인의 네임서버 정보
- SOA(Satrt Of Authority) : 존의 시작을 표현하는 레코드로 존의 기본 속성 정보 버전, 존 전송 주기등을 담고 있음
- HINFO(Host Information) : 호스트의 CPU, OS 정보
- CNAME(Canonical Name) : 호스트의 별명
- TXT(Text) : 도메인에 대한 텍스트 정보로 다양한 용도로 사용 됨 -> SPF 레코드 설정, DKIM을 사용하여 디지털 이메일 서명 추가, DMARC를 사용하여 발신 스팸 방지
- PTR(Pointer) -> 리버스 존 파일에 설정하는 레코드로 A 레코드와는 반대로 IP 에 대한 도메인 정보를 담고 있음

* 재귀적 질의에 대한 제한 (Recursive Query) : 악의적으로 공격자에 의한 과도한 재귀적 질의 요청 발생시 Dos 형태의 공격으로 악용될 소지가 있으며 DNS Cache Poisoning 공격에 노출 될 수 있으므로 적절한 접근 제어가 필요함
-> allow-recursion을 이용하여 설정함 (none 또는 no 로 설정하면 재귀적 질의를 허용하지 않음) , IP 또는 IP 대역, 별도의 acl 을 정의하고 allow-recursion에 acl명을 명시하여 제한적으로 질의를 허용할 수 있음

```
recursion no;
allow-recursion {none;};


allow-recursion {127.0.0.1; 192.168.56.0/24;};
allow-recursion {internal};
```

-> 설정파일명 : /etc/named.conf

* 존 전송에 대한 제한 : 악의적인 공격자에 의해 Dos 공격으로 악용 될 수 있음 -> 지속해서 존 전송을 요청하여 네트워크 및 시스템 자원을 소모시킬 수 있고 zone 데이터에 저장된 해당 도메인의 서버 정보가 그대로 노출될 수 있음
-> Master DNS 서버만 운영할 경우에는 존 전송을 허용하지 않고 Master 와 Slave 를 운영하는 환경에서는 지정한 Slave 에서만 존 전송을 요청하도록 설정해야 함
-> 존 전송을 허용하지 ㅏ 않을 경우 allow-transfer을 none 으로 설정하고 slave 만 허용할 경우 allow-transfer 에 slave의 주소 정보를 입력함

```
allow-transfer {none;};
allow-transfer{slave ip;};
```

-> 설정파일명 : /etc/named.conf

-> dig 명령의 axfr/ixfr 질의 유형을 통해 존 전송 허용 여부를 테스트할 수 있음

* HTTP : 웹 상에서 클라이언트와 서버 간에 통신을 위해 개발된 프로토콜 , 주로 80/tcp 사용, 클라이언트 요청과 서버 응답으로 이루어져 있음
cf. 하이퍼텍스트 문서 : 참조 또는 링크를 통해 한 문서에서 다른 문서로 접근할 수 있는 문서. 대표적으로 HTML이 있음

* HTTP/1.1 버전부터 Connection 헤더에 Keep-Alive 옵션 추가 -> TCP 연결 상태를 웹 서버 설정에 따라 일정 시간 지속시키는 옵션으로 한 번의 연결 이후에 요청/응답을 반복할 수 있음 -> TCP 연결 설정 및 종료에 대한 부하를 줄이면서 효율적으로 클라이언트 요청을 처리할 수 있음

* HTTP는 상태정보를 유지하지 않는 프로토콜 (Stateless) -> 상태정보를 유지하지 않는다는 것은 동일 클라이언트의 현재 요청과 이전요청을 식별하지 못한다는 의미 -> 현재 연결에 대한 클라이언트의 어떠한 상태정보도 유지하지 못한다는 의미 -> 쿠키/세션을 사용해서 클라이언트의 상태정보를 유지함

* 쿠키 : 클라이언트의 상태정보 기술중의 하나로, 개별 클라이언트 상태 정보를 HTTP 요청/응답 헤더에 담아서 전달하는 작은 정보/데이터를 의미한다.

-> 서버에서 Set-Cookie 응답헤더를 통해 쿠키를 설정하여 클라이언트로 전달하면 클라이언트는 Cookie 요청헤더를 이용해 지속해서 쿠키를 전달하는 형태로 동작한다 -> 영속 쿠키(클라이언트에 파일형태로 지속해서 존재하는 쿠키로 쿠키에 설정된 사이트 요청시마다 Cookie 요청 헤더에 쿠키 정보를 담아서 전달 함)/세션 쿠키(클라이언트 메모리상에 세션이 유지되는 동안 존재하는 쿠키로 세션이 종료되면 소멸함)가 있다.

-> 쿠키의 보안 취약점 : 클라이언트의 상태정보를 클라이언트에 저장하고 HTTP 요청/응답 헤더에 담아서 전달하기 때문에 해킹 및 스니핑 공격에 의한 변조와 외부 노출에 취약한 특성이 있음. 중요 정보를 저장할 때는 세션을 이용하는 것이 권장되며 쿠키를 사용할 경우에는 암호화를 적용해야 함

* 세션 : 개별 클라이언트 상태정보를 서버에 저장하는 기술을 의미함. 서버는 개별 클라이언트 세션을 식별하기 위해 session ID 를 부여하고 세션 ID는 세션쿠키를 이용하여 클라이언트와 서버 간에 주고 받음. 클라이언트 상태 정보가 서버에 저장 되기 때문에 쿠키에 비해 안전함

cf. HTTP 세션 하이재킹 (HTTP Session Hijacking) : 공격자가 정상적인 사용자의 세션 ID 정보를 탈취하여 정상 사용자로 위장한 접근을 하는 것

* HTTP 쿠키 관련 보안 속성
- httponly : Set-Cookie 응답 헤더 (response header) 에 설정하는 속성으로 클라이언트에서 스크립트를 통해 해당 쿠키에 접근하는 것을 차단해주는 속성 , XSS 공격에 대응하기 위해 사용함

- secure : Set-Cookie 응답 헤더에 설정하는 속성으로 클라이언트에서 HTTPS 통신일 경우에만 해당 쿠키를 전송하고 HTTP 통신일 경우에는 전송하지 않는 속성 -> HTTPS 만 쿠키를 사용할 수 있게 하는 것 , 전송중에 평문 쿠키가 노출되는 것을 방지하는 목적으로 사용함 -> 쿠키의 기밀성 보장 (Confidentiality)

* HTTP 요청 메시지 구조 : Request Line(요청 메소드, 요청 URI, HTTP 버전 정보), Request Header(Host, User-Agent, Referer), Empty Line(헤더의 끝을 의미하는 개행), Message Body(클라이언트에서 서버로 전달하는 데이터를 담는 부분. GET 방식은 요청 데이터가 없으므로 메시지 바디가 없음)

cf. 개행 : CRLF (Carriage Return Line Feed) -> 0x0d0a

* HTTP 주요 요청 메소드
- GET : 요청 URI로 지정한 자원을 서버에 요청하는 메소드로 요청 메시지 바디가 없으며 필요시 쿼리스트링을 이용하여 제한된 데이터 전송이 가능함 -> GET 방식의 쿼리 스트링으로 전달하는 데이터는 요청 URI 끝에 추가해서 전달하기 때문에 주소창에 정보가 쉽게 노출되고 서버 액세스 로그 상에 그대로 남아 있다. 따라서 중요 정보를 GET으로 전달하는 것은 보안상 매우 취약하므로 POST 방식으로 전달하는 것을 권장 한다
- POST: 요청 URI로 지정한 자원에 데이터를 전달하여 이를 처리한 결과를 서버에 요청하는 메소드로 요청 메시지 바디를 포함한다.
- HEAD : GET 메소드와 유사하게 요청하지만 서버 응답시에 응답 메시지 바디를 제외하고 헤더부분만 응답해주는 메소드로 요청 자원에 대한 처리는 서버에서 이루어지지만 그 결과로 헤더값만을 전송함. URL/링크의 유효성을 검증하기 위한 목적으로 사용함
- OPTIONS : 서버가 지원하는 메소드를 확인하는 목적으로 사용하는 메소드 -> 서버 reponse 에서 Allow 응답 헤더를 통해 웹서버에서 지원하는 메소드 정보를 전달 함
- CONNECT : 클라이언트와 서버간에 터널링 목적으로 사용하는 메소드로 웹 서버가 프록시 역할을 수행함
- PUT : 요청 메시지 바디에 포함되어 있는 데이터를 요청 URI로 지정한 자원으로 저장하도록 하는 메소드
- TRACE : 클라이언트로부터 수신한 메시지를 서버에서 그대로 반환하는 메소드로 요청에 대한 루프백 테스트 용도로 사용함
- DELETE : 요청 URI로 지정한 자원을 서버에서 삭제하도록 하는 메소드

* HTTP 응답 메시지 구조 : Status Line(HTTP 버전, 상태 코드, 응답 구문), Response Header(Content-Type/Content-Length), Empty Line(헤더의 끝을 의미하는 개행), Message Body(메시지 내용, 서버에서 클라이언트로 전송하는 데이터를 담는 부분)

* HTTP 응답 주요 상태 코드
- 1xx : 100 -> Continue(클라이언트로 부터 일부 요청을 받았으며 나머지 정보를 계속 요청함)
- 2xx : 200 -> OK(요청이 성공적으로 수행 됨), 201 -> Created(PUT메소드에 의해 원격지 서버에 파일이 생성됨, 202 -> Accepted(웹 서버가 명령 수신함)
- 3xx : 301 -> Moved Permanently (요청 자원의 위치가 영구적으로 변경됨, Location 응답 헤더를 통해 자원의 변경된 URL 정보를 전달함) , 302 -> Found (요청 자원의 위치가 임시적으로 변경 됨. Location 응답 헤더를 통해 자원의 변경된 URL 정보를 전달함), 304 -> Not Modified (요청한 자원이 변경되지 않았으므로 클라이언트 로컬 캐시에 저장된 자원을 이용하라는 의미)
- 4xx : 400 -> Bad Request (요청 메세지 문법 오류), 401 -> Unauthorized (요청한 자원에 대한 인가 필요, 요청 자원을 실행하는데 필요한 적절한 권한이 없음을 의미함), 403 -> Forbidden(요청한 자원에 대한 접근 차단), 404 -> Not Found (요청한 자원이 존재하지 않음)

- 5xx : 500 -> Internal Server Error (내부 서버 오류)

* FTP (File Transfer Protocol) : TCP/IP 기반의 서버와 클라이언트 사이에 파일 전송을 위한 통신 프로토콜 중 하나임, FTP 서비스는 아이디 및 패스워드가 암호화되지 않은 채로 전송되어 간단한 스니퍼에 의해서도 아이디 및 패스워드가 쉽게 노출될 수 있으므로 반드시 필요한 경우를 제외하고는 사용하지 않는 것이 좋음, 암호화된 통신을 제공함 -> SFTP (SSH File Transfer Protocol , 22/tcp), FTPS(FTP over SSL/TLS, 990/tcp)

* FTP 서비스의 동작 방식 : 제어채널(control channel), 데이터 채널(data channel)의 생성 방식에 따라 능동모드(active mode) 와 수동 모드 (passive mode) 로 구분 가능

- 능동 모드 : 기본 동작 모드 -> 제어 채널은 클라이언트에서 서버의 21/tcp 로 접속하여 제어 채널을 생성한 후 명령을 전달함, 데이터 채널은 서버에서(20/tcp) 클라이언트의 1024/tcp 이상 포트로 접속하여 데이터 채널을 생성한 후 데이터 파일을 전달함 (1024/tcp 이상 포트는 클라이언트에서 PORT 명령어를 사용해서 서버에 요구함)

=> 클라이언트에서 서버로 제어 채널을 생성하고, 서버에서 클라이언트로 데이터 채널을 생성함
=> 클라이언트에 방화벽이 설치되어 있어 원격 FTP 서버에서 클라이언트로의 연결이 불가능하다면 데이터 채널 접속이 불가능하여 파일,파일목록등을 받을 수 없는 문제가 발생할 수 있음 -> 클라이언트 방화벽에 원격 FTP 서버에서 클라이언트로의 접속을 허용하거나, 수동모드로 원격 FTP 서버에 접근해야 함

- 수동 모드 : 클라이언트에서 서버의 21/tcp 포트로 접속하여 제어 채널을 생성한 후 클라이언트에서 서버의 1024/tcp 이상 포트로 접속하여 데이터 채널을 생성한 후 데이터 파일을 전달함 -> 능동 모드와 다르게 제어, 데이터 채널 모두 클라이언트에서 서버로 연결 됨 (1024/tcp 이상 포트는 클라이언트에서 PASV 명령어를 사용해서 서버에 요구함)

=> FTP 서버는 데이터 채널 생성을 위해 인바운드 트래픽에 대한 방화벽의 1024/tcp 이상 포트를 모두 허용해야 하는 문제점이 있음 -> 방화벽의 상태검사 기능을 통해 연결 상태를 추적하여 데이터 채널을 허용하도록할 수 있음

cf. 방화벽의 상태 검사 기능 (stateful inspection) : 방화벽을 통과하는 모든 트래픽에 대해 프로토콜별 연결 상태를 추적하여 패킷 필터링을 수행하는 방화벽 기능을 말함, TCP 는 제어 채널 포트인 21/tcp 만 허용해주면 데이터 채널을 자동으로 허용해 줌

* FTP 바운스 공격 : 제어 채널과 데이터 채널이 분리되어 있고, 데이터 채널을 생성할 때 임의의 목적지 IP와 PORT 번호를 설정할 수 있는(목적지를 확인하지 않음) FTP 설계의 구조적 취약점을 이용하여 공격 대상 네트워크를 스캔하거나 공격자가 원하는 서버로 데이터를 전송하는 형태의 공격을 의미함 -> FTP 설계의 구조적 취약점 이용, 주로 익명 FTP 서버를 이용하며 'PORT' 명령의 목적지 IP 와 Port 번호를 조작하여 FTP 서버가 공격자가 원하는 목적지 호스트와 데이터 채널을 연결하도록 시도함, 이를 통해 공격자는 FTP 서버를 거쳐 외부에서 접근할 수 없는 내부 네트워크를 스캔하거나 공격자가 원하는 내부 서버로 데이터를 전송하는 등의 악의적인 행위를 할 수 있음

* TFTP(Trivial FTP) 공격 : TFTP는 읽기 전용 메모리나 디스크가 없는 워크스테이션에 설치될 수 있을 정도로 매우 단순하고 간단한 파일 송수신 프로토콜로 69/udp 포트를 사용함. 별도의 인증과정 없이 지정한 디렉터리에 접근하여 파일 송수신을 할 수 있는 보안상 취약점이 존재함
-> 자체 디스크가 없는 워크스테이션에 부팅 이미지를 전달하거나 네트워크 장비의 운영체제를 백업 또는 복구하거나 설정 정보를 저장하거나 가져오는 용도로 사용
-> 보안 기능이 없는 TFTP 의 취약점을 이용하여 공격자가 TFTP 서버에 있는 파일에 접근하거나 악의적인 파일을 생성하는 공격을 의미함

=> 대응책 : TFTP 서비스가 불필요한 경우 제거 필요, 사용해야 한다면 secure mode로 운영 (chroot 기능을 이용하는 것으로 지정한 디렉터리를 최상위 디렉터리로 지정하여 지정한 디렉터리의 상위 디렉터리로 접근하지 못하도록 제한하는 방식을 의미함)

* 익명 FTP 공격 (annoymous FTP attack) : 익명 FTP 서비스는 익명 계정으로 FTP 접속이 가능한 서비스를 말한다. 익명 계정이란 ID가 annoymous 고 비밀번호가 없는 계정을 의미한다 -> 익명 FTP 공격은 비밀번호를 요구하지 않는 익명 FTP 서비스의 취약점을 이용하여 공격자가 FTP 서버에 있는 파일에 접근하거나 악의적인 파일을 생성하는 공격을 의미한다.
-> 보안상 심각한 취약점을 가지고 있으므로 사용하지 않는게 가장 좋음 : vsftpd.conf 파일에서 annoymous_enable=NO 로 설정하도록 한다.

* FTP 접근 제어 설정 : FTP 서비스는 기본적으로 평문 송수신을 하므로 아이디 및 패스워드가 노출되기 쉽다. 따라서 중요 계정은 FTP 접속을 제한해야 한다 -> ftpusers(접속을 제한할 계정 정보를 담고 있는 설정 파일)에 root 등 중요 계정을 명시한다. vsFTP서버의 경우 'user_list' (vsftp.conf 설정파일에서 먼저 userlist_enable=YES 처리를 한 뒤 user_list 를 생성하서 접속을 제한할 계정명을 나열함) 파일을 사용해서 계정별 FTP 접속 제한을 설정할 수 있다. 또는 TCP Wrapper 을 이용한 IP 접근제어를 사용한다. hosts.allow 파일과 hosts.deny 파일을 이용할 수 있다. (vsFTP 서버의 경우 vsftp.conf 파일의 tcp_wrappers=YES 항목에 설정 후 사용 가능)

* SNMP (Simple Network Management Protocol)
- 실시간으로 상태를 모니터링 및 설정할 수 있는 서비스로 SNMP는 프로토콜일 뿐이며 이를 활용하여 실제 네트워크 관리 정보를 얻기 위해서는 관련 프로그램이 준비되어야 함
- 관리의 편의성도 있지만 여러 취약점이 존재하여 서비스거부공격(Dos), 버퍼 오버플로우, 비인가 접속 등 여러 문제점이 발생할 수 있음
- SNMP 구성 요소 : 관리 시스템 (Manager) -> Agent 에 필요한 정보를 요청하는 모듈 / 관리 대상 (Agent) -> 관리 대상 시스템에 설치되어 필요한 정보를 수집하고 Manager 에게 전달해주는 역할을 하는 모듈
- 네트워크 관리를 위한 목적으로 주로 서버나 네트워크 장비에서 SNMP를 설정한 후 MRTG 프로그램을 이용하여 트래픽 관리등을 위해 사용 됨
- Application 계층 프로토콜로 메시지는 단순히 요청과 응답 형식의 프로토콜에 의해 교환되기 때문에 전송계층 프로토콜로는 UDP 를 사용함

-> 보안상 취약함, Manager - Agent 구조, Application 프로토콜, UDP 사용

cf. MRGT(Multiple Router Traffic Grapher) : SNMP 기반의 장비 모니터링 프로그램 -> 네트워크 사용량 모니터링

* SNMP 동작 방식 : Manager(관리시스템) 은 162/udp 포트를 사용하며 Agent(대행자)는 161/udp 포트를 이용한다 -> Manager 와 Agent 가 통신하기 위해서는 각각의 SNMP 버전, Community String, PDU 가 일치 해야함

cf. PDU : Protocol Data Unit 의 준말로 통신하기 위한 메시지유형

* PDU 타입 : Get request (Manager -> Agent 원하는 객체의 특정 정보를 요청) , Get Next Request (Manager -> Agent 이미 요청한 정보의 다음 정보 요청), Set Request (Manager -> Agent 로 특정한 값을 설정하기 위해 사용), Get Response (Agent -> Manager 로 해당 변수값 전송) , Trap (Agent 가 Manger 에 어떤 정보를 비동기적으로 알리기 위해 사용함 . notify 라고 하며 callback 함수와 같은 역할을함, 유일하게 비동기적임) , Get Bulk Request (SNMPv2에서 추가됨, 요청할 객체의 범위를 지정하여 한번에 요청 Manager -> Agent) , InformRequest (SNMPv2에서 추가 됨. 관리시스템 간의 정보 전달 목적으로 사용)

* SNMP 데이터 수집 방식 : Polling 방식 (Manager가 Agent에 정보를 요청하면 응답해주는 방식으로 Agent 가 161/udp 사용) / Event Reporting 방식 (Agent 가 이벤트 발생시 Manager 에게 알리는 방식으로 Trap 메시지를 통해 이벤트 발생 여부를 전달함. Manager가 162/udp 사용)

* SNMP 주요 용어
- MIB (Management Information Base) : 관리되어야 할 특정한 정보, 자원을 객체라 하고 이러한 객체들을 모아 둔 집합체 -> 관리자가 조회하거나 설정할 수 있는 객체들의 데이터 베이스
- SMI (Structure Management Information) : MIB 를 정의하기 위한 일반적인 구조, ASN.1 언어를 사용함 -> 정의된 모든 객체는 name(Object Indentifier), syntax (객체 데이터 유형), encoding (ASN.1 의 encoding rule 중 BER) 을 가짐
- community string : SNMP 데몬서버와 클라이언트가 데이터를 교환하기 전에 인증을 위해 사용되는 일종의 패스워드. 초기값으로 public 또는 private 를 사용함. -> 초기값을 사용하는 것은 패스워드를 사용하지 않는 것 보다 위험하므로 초기값 변경이 꼭 필요함 (추측하기 어렵고 의미 없는 문자열) , SNMP 에는 Read Only 줄여서 RO 와 ReadWrtie, 줄여서 RW 를 모드를 제공함. 쓰기 권한이 있으면 주요 설정을 수정할 수 있는 등 심각한 보안 문제를 유발할 수 있으므로 가급적 사용을 자제 해야 함

* SNMP 접근 제한 설정 : SNMP는 트래픽 정보 뿐만 아니라 각종 하드웨어 정보까지 제공하므로 보안에 주의해야함 -> 읽기 권한 뿐아니라 쓰기 권한까지 있으면 설정 파일을 열람하거나 직접 네트워크 설정을 변경할 수 있음

* SNMPv1 : 보안 기능이 없으며 community string 만 일치하면 모든 정보를 얻을 수 있음 -> 1988년 IAM 에서 표준화 작업을 거쳐 SGMP(Simple Gateway Monitoring Protocol) 를 발전시킨 프로토콜, 클라이언트와 데몬 (여기서 데몬은 Manager, 클라이언트는 Agent 를 의미함) 간의 통신이 평문으로 전송되므로 스니핑에 취약함

* SNMPv2 : 클라이언트와 데몬 (여기서 데몬은 Manager, 클라이언트는 Agent 를 의미함) 간의 통신이 평문으로 전송되므로 스니핑에 취약함 , 버전 1 의 문제점을 해결하기 위해 전송하는 정보에 대한 암호화와 해시를 추가했으나 송신처 인증 기능이 없음 -> 가장 많이 사용되는 SNMPv2c 는 SNMPv2에 복잡한 보안 기능을 제거한 버전으로 보안상 취약함

* SNMPv3 : 데이터 인증, 암호 기능 및 재사용 방지, 세분화된 접근통제 등 개선된 보안서비스를 제공함으로써 이전 SNMP 버전에서 제공되지 않았던 안전한 통신망 관리를 위한 기반 기술을 제공함 , SNMPv2 에 비해 계정과 암호로 인증하는 보안 기능이 추가 됨

* SNMPv3 보안 서비스 : 비인가된 사용자에 의한 데이터의 변경(무결성 침해), 도청 (기밀성 침해), 재사용 공격에 대응하는 기능을 제공하는 '사용자 기반 보안모델 (USM : User Security Model)과 인가된 사용자의 MIB 접근 통제 기능을 제공하는 ' 뷰 기반 접근 통제 모델 (VACM : View-based Access Control Model' 에 의해 제공 됨.

Authoriative 엔진은 SNMP 명령을 처리하거나 통지를 발생시키는 SNMP 엔진을 의미하며 일반적으로 SNMP 에이전트 기능을 수행하는 엔진을 나타냄

-> msgSecurityParameter (보안 매개변수) : SNMPv3 부터 공격 위협으로부터 방어하기 위한 보안 매개변수들을 사용함 -> 재전송 공격 방지 (Authoritative 엔진 ID, Authoritative 엔진부트횟수, Authoriative 엔진시각) ,위장공격과 메시지 위변조 공격 방지 (사용자, 인증 매개변수), 암호 매개변수(도청/스니핑 공격, 정보 노출 공격 방지, DES-CBC를 통한 메시지 암호화 지원)

* NMS (Network Management System) : 네트워크상의 자원들을 모니터링하고 제어하기 위한 도구. 전체 시스템에서 관리하는 네트워크의 요소의 각 지점과 특정한 속성에 주소와 이름을 지정하고 주기적으로 각 요소가 가진 정보를 중앙 제어 센터에 제공하는 구조로 되어 있음 (Manager - Agent 구조)

-> 네트워크 모니터링 방식은 Polling 방식(요청/응답 방식으로 Manager 가 원하는 정보를 Agent에 요청하면 Agent가 MIB로부터 정보를 추출하여 응답하는 방식, 주기적인 폴링으로 Agent의 상태변화 정보를 얻을 수 있음)과 Event Reporting 방식(Agent가 자신의 상태를 주기적으로 Manager에게 알리는 방식, 중요한 상태변화나 결함과 같은 예상치 못한 사건이 발생하는 경우 Polling보다 빠르게 파악할 수 있음)으로 나뉨

* DHCP(Dynamic Host Configuration Protocol) : 동적으로 클라이언트의 네트워크주소를 설정하기 위한 프로토콜, 67/UDP(server), 68/UDP(client) 포트 사용

- ipconfig /release : 할당받은 IP주소를 해제함
- ipconfig /renew : 새로운 IP주소를 DHCP 서버로부터 받음

-> IP 할당 절차 : DHCP Discover 메시지 (DHCP서버를 찾기 위한 메시지로 자신의 MAC 정보를 담아서 브로드캐스트함) , DHCP Offer메시지 (DHCP 서버가 클라이언트에게 IP를 제공해주는 메시지), DHCP Request 메시지 (해당 IP를 사용하겠다고 서버에게 요청하는 메시지, 서버가 Offer 메시지를 통해서 주소를 제공하면 바로 설정하는 것이 아닌 사용하겠다는 메시지를 다시 보냄), DHCP Ack 메시지 (서버는 해당 MAC과 IP정보를 테이블에 저장하고 다시 한번 할당된 주소정보를 클라이언트에게 전송해줌. 클라이언트는 이 메시지를 받고 IP를 정함)

-> DHCP Starvation 공격 : DHCP 서버의 할당 가능한 IP를 모두 소진하게 만들어 IP 할당이 불가능하게하는 공격으로 discover 메세지를 서로 다른 MAC 주소로 대량으로 보내게 되면 이에 대한 offer가 올 것이고 여기서 request 메시지까지 보낸 후에 실제로는 할당하지 않음

* SQL Injection 취약점 : Web Application에서 입력받아 데이터베이스로 전달하는 정상적인 SQL질의를 변조, 삽입하여 불법 로그인, DB 데이터 열람, 시스템 명령 실행 등을 수행하여 비정상적인 데이터베이스 접근을 시도하는 공격 기법 -> DB에 악성 스크립트를 삽입하여 접근하는 사용자들을 피싱 사이트 또는 악성 코드 유포 사이트로 유도함, Stored Procedure를 통한 OS 명령어 실행, DB에 있는 개인정보 획득
cf. SQL Injection 스캐너 : Nikto, SQLMap, Absinthe

* SQL Injection 취약점 - 공격 방식에 의한 분류
- Form SQL Injection : HTML Form 기반 인증을 담당하는 애플리케이션의 취약점이 있는 경우 사용자 인증을 위한 질의문 조건을 임의로 조작하여 인증을 우회하는 기법 -> 질의문의 조건절이 항상 참이 되도록 조작, 반환되는 레코드셋의 첫 번째 레코드에 해당하는 사용자 권한을 획득하게 됨 -> ' or 1=1#, ' or '1 = '1'#, ' 'a'='a'#

cf. URL 인코딩 : URL 상에서 영문자, 숫자, 일부 특스문자를 제외하고 사용할 수 없는 문자를 URL상에 안전하게 표현하기 위한 인코딩 방식을 말하며 Percent Encoding 방식을 말한다.

=> mysql_real_escape_string() 함수를 통해 특수문자 이스케이프 처리를 할 수 있음 -> 이것을 통해 SQL Injection 취약점을 대응할 수 있음
=> Prepared Statement 를 사용한다 (선처리 질의문 : 외부로부터 입력값을 제외한 질의 부분을 미리 컴파일하여 생성한 후 반복적으로 입력값만을 매개변수로 전달하여 질의문을 실행 함, 반복적인 컴파일 과정을 수행하지 않고 사전에 컴파일된 질의문을 사용하기 때문에 성능상의 장점과 함께 공격자가 악의적인 입력값을 삽입하더라도 매개변수로 바인드할 뿐 질의문 자체는 영향받지 않아 SQL Injection 공격을 방어하는 효과가 있다.)
=> 파라미터 필터링을 통한 SQL Injection 방지 : 문자열을 필터링할 수 있는 사용자 정의 함수를 생성하여 파라미터에 대한 검증을 수행함. 블랙리스트 문자열을 포함할 경우 오류 메시지를 반환하여 질의를 수행하지 않도록 한다.

- Union SQL Injection : Union 질의를 이용하여 질의의 결과를 다른 질의의 결과에 결합하여 공격하는 기법 (각각의 select 문의 필드 개수가 같아야 하며 필드 타입이 호환할 수 있어야 함 -> 컬럼 개수 파악을 위해 order by 절에 지정한 인덱스의 컬럼이 없으면 오류가 발생한다는 것과 union select문을 이용하면서 컬럼 개수가 맞지 않으면 오류가 발생한다는 특성을 이용 할 수 있다)

=> SQL Injection에 대한 대응방식은 사용자 입력값에 대한 검증을 통해 불필요한 문자열의 입력을 막는 방식으로 대응해야 함

- Stored Procedure SQL Injection : 저장 프로시저는 일련의 질의를 마치 하나의 함수처럼 실행하기 위한 질의의 집합을 의미한다.
- Mass SQL Injection : 기본 SQL Injection이 확장된 개념으로 한 번의 공격으로 대량의 DB값이 변조되어 홈페이지에 치명적인 영향을 미치는 공격 ->DB값 변조시 악성 스크립트를 삽입하여 사용자가 변조된 사이트 방문시 감염되거나 악성 Bot이 설치 됨 (js, swf, exe 파일 사용)

* SQL Injection 대응 방안 : 데이터베이스와 연동하는 스크립트의 모든 파라미터를 점검하여 사용자의 입력값에 SQL 질의문에 이용되는 특수문자 및 select,update,insert,delete union 등의 SQL관련 문자열을 필터링 함, 입력되는 문자열의 길이를 제한하고 숫자일 경우 숫자인지 체크하는 함수를 사용함, MS-SQL인 경우 master..xp_cmdshell, xp_startmail, xp_sendmail 과 같은 확장 프로시저는 운영체제 명령 실행, SQL 인젝션 등에 이용 되기 때문에 제거, 웹 애플리케이션이 사용하는 데이터 베이스 사용자 권한을 제한함, 선처리질의문(Prepared Statement)은 사전에 컴파일된 질의문을 이용하는 방식으로 이후 입력되는 사용자 입력은 질의문 자체를 변경할 수 없고 별도의 매개변수로 전달되어 실행되기 때문에 공격자가 악의적인 입력값을 삽입하더라도 SQL 질의문 자체에 영향을 미치지 않는다.

* SQL Injection 취약점 - 공격 유형에 대한 분류
- Error-Based SQL Injection(에러기반 SQL Injection) : DB질의에 대한 에러값을 기반으로 한 단계씩 점진적으로 DB정보를 획득하는 방법임. 클라이언트에서 조작할 수 있는 파라미터에 SQL질의 관련 특수문자를 삽입하여 SQL 에러가 발생한다면 해당 취약점이 있다고 판단함 -> DB 질의에 대한 에러가 외부로 노출되는 취약점을 이용한 공격임. 최근에는 잘 없음

- Blind SQL Injection : 오류 메시지가 아닌 질의 결과의 참과 거짓을 통해 의도하지 않은 SQL문을 실행함으로써 데이터베이스를 비정상적으로 공격하는 기법 -> Error-Based에 비해 많은 비교 과정을 거치기 때문에 취약점 발견시 자동화 도구를 이용하는 것이 일반적임

* XSS (크로스 사이트 스크립트 : Cross Site Script) 취약점 : 웹 애플리케이션에서 사용자 입력값에 대한 필터링이 제대로 이루어지지 않을 경우 공격자가 입력이 가능한 폼에 악의적인 스크립트를 삽입, 해당 스크립트가 희생자 측에서 동작하도록 하여 악의적인 행위를 수행하는 취약점을 말한다. -> 악의적인 스크립트 삽입, 동작은 희생자 측에서 -> XSS 취약점이 발생할 수 있는 입력 가능한 폼, URL 파라미터 등에 스크립트를 삽입하여 해당 스크립트가 동작하면 XSS에 취약하다고 판단한다.

- Stored XSS (저장형 XSS) : 공격자가 취약한 웹서버에 악성 스크립트를 저장해놓으면 희생자가 해당 자료를 요청할 때 해당 악성 스크립트가 삽입된 응답 페이지가 전달되어 클라이언트 측에서 동작하는 방식, htmlspecialchars() 함수는 html문법상 특별한 의미가 있는 특수문자의 기능을 제거하고, 일반문자로 처리해주는 함수로 HTML Entity로 변환하여 출력해준다.
- Reflected XSS (반사형 XSS) : 외부에 있는 악성 스크립트가 희생자 액션에 의해 취약한 웹서버로 전달되고 웹서버의 응답 페이지에 해당 악성 스크립트가 삽입되어 희생자 측에서 동작하는 방식, Stored XSS는 악성 스크립트가 취약한 웹서버의 DB에 저장된 형태임에 반하여 Reflected XSS는 DB에 저장된 것이 아니라 악성 스크립트가 포함된 요청 정보를 처리하는 과정에서 악성 스크립트가 포함된 응답 페이지가 생성되어 희생자에게 전달되는 방식을 의미함.
- DOM based XSS (DOM 기반 XSS) : 희생자의 웹브라우저에서 응답 페이지에 포함된 자바스크립트가 동작하면서 DOM 객체를 실행할 때 URL 등에 포함된 악성 스크립트가 동작하는 방식 -> 저장형 및 반사형 XSS의 경우 응답 페이지에 악성 스크립트가 포함되어 웹브라우저로 전달되어 실행되는 반면에 DOM 기반의 XSS는 응답 페이지와 관계없이 웹브라우저에서 발생한다는 차이점이 있음

cf. DOM은 문서 객체 모델로 웹페이지 내에 있는 모든 객체를 조작, 관리할 수 있는 계층 구조 형태의 모델을 말한다. DOM을 통해서 HTML 문서를 계층적으로 보면서 컨텐츠를 동적으로 변경할 수 있다.

* XSS 대응책 : 사용자 입력값에 대한 검증은 반드시 서버단에서 해야함, 사용자 입력 문자열에서 HTML 코드로 인식될 수 있는 특수 문자를 일반문자(HTML Entity)로 치환하여 처리한다, 게시판 등에서 HTML 태그를 허용해야만 하는 경우라면 HTML 태그 화이트리스트를 선정한 후 해당 태그만 허용하는 방식을 적용 함

cf. 웹 프록시 도구 : 웹을 이용한 해킹 공격에 가장 활발히 사용되는 도구로 클라이언트가 요청한 HTTP 요청/응답 정보를 중간에서 가로채 필터링을 우회하거나 서버에 전송되는 데이터를 조작해 허가되지 않은 곳의 정보를 훔쳐보거나 변경, 삽입 등을 가능하게 함. 파로스(Paros), 버프(Burp) 등이 해당 됨

cf. HTML 특수 문자와 HTML Entity
< : <
> : >
(: (
) :)

& : &
/ : &#x2F
“ : "
‘ : '
:

* 크로스 사이트 요청 위조 (CSRF : Cross Site Request Forgery) : 웹 애플리케이션에서 정상적인 경로를 통한 요청과 비정상적인 경로를 통한 요청을 서버가 구분하지 못할 경우 공격자가 스크립트 구문을 이용하여 정상적인 사용자가 조작된 요청을 전송하도록 하여 게시판 설정 변경, 회원 정보 변경 등의 문제가 발생할 수 있는 취약점을 의미함, 희생자 모르게 공격자의 의도된 행위를 수행하게 하여 특정 피해를 주는 형태, 공격당한 사용자의 권한을 공격자가 그대로 사용하게 되므로 사용자의 권한 수준에 따라 그 피해 범위가 달라질 수 있음

-> CSRF 취약점이 발생할 수 있는 입력 가능한 폼에 img 태그의 src 속성을 이용하여 조작된 요청을 입력, 요청이 실행되면 취약한 것으로 판단 함

* 크로스 사이트 요청 위조, CSRF 대응책 : 웹 애플리케이션에 존재하는 모든 HTTP 요청 내에 예측할 수 없는 임의의 토큰을 추가하여 정상적인 요청과 비정상적인 요청을 판별하는 것을 권장 함 , 또한 중요한 기능에 대해서는 사용자 세션 검증과 더불어 재인증을 유도한다, XSS와 공격 방식이 유사하므로 XSS에 대한 취약점 제거가 필요하다.

* 서버 사이드 요청 위조 (SSRF : Sever Side Request Forgery) : 적절한 검증 절차를 거치지 않은 사용자 입력값을 서버 간의 요청에 사용하여 악의적인 행위가 발생하는 취약점으로 외부에 노출된 웹서버에 취약한 애플리케이션이 존재하는 경우 공격자는 URL 또는 요청 정보를 위조하여 접근 통제를 우회하는 방식으로 보호된 내부 서버에 있는 정보를 획득하거나 비정상적인 행위를 유도할 수 있음, 공격자가 조작된 요청을 웹서버에 전송하여 웹서버가 내부 네트워크에 있는 다른 서버에 악의적인 요청을 보내는 취약점임

-> CSRF(Cross Site Request Forgery) 는 공격자가 정상적인 사용자의 요청 정보를 조작하여 웹 서버에서 사용자의 권한으로 정보 조작 등의 악의적인 행위를 수행하는 취약점을 말함 -> 조작된 요청 발생 주체는 사용자임

-> SSTF(Server Side Request Forgery) 는 공격자가 요청 정보를 조작하여 웹서버가 내부 서버에 조작된 요청을 하도록 하여 정보 탈취 등의 악의적인 행위를 수행하는 취약점을 말함 -> 조작된 요청 발생 주체는 웹 서버임 (웹 애플리케이션)

* SSRF 대응책 : 사용자 입력값을 다른 시스템의 서비스 호출에 사용하는 경우 사용자의 입력값을 화이트리스트 방식으로 필터링 함, 동일한 내부 네트워크에 있는 서버 간이라도 기기 인증, 서비스 접근 권한 등을 확인하여 요청을 처리 함

* 운영체제 명령 실행 (OS Command Excution) 취약점 : 적절한 검증 절차를 거치지 않은 사용자 입력값이 운영체제 명령어의 일부 또는 전부로 구성되어 실행되는 경우, 의도하지 않은 시스템 명령어가 실행되어 부적절하게 권한이 변경되거나 시스템 동작 및 운영에 악영향을 미칠수 있는 취약점을 말한다 = 운영체제 명령어 삽입 보안 약점 = 웹 애플리케이션에서 system(), exec() 와 같은 시스템 명령어를 실행할 수 있는 함수를 제공하며 사용자 입력값에 대한 필터링이 제대로 이루어지지 않을 경우 공격자가 시스템 명령어를 호출할 수 있는 취약점을 말함 -> 공격자는 취약점을 이용하여 시스템 계정 정보 유출, 백도어 설치, 관리자 권한 탈취 등을 진행할 수 있음

-> 입력가능한 폼에 명령어를 삽입하여 해당 명령어가 실행되면 취약한 것으로 판단함 -> URL 파라미터에 대해 리눅스/유닉스에서는 ls, cat 윈도우에서는 dir, config 등의 명령어를 삽입해봄

* 운영체제 명령 실행 취약점 대응책 : 사용자 입력값에 대해 운영체제 명령어를 실행할 수 있는 문자열을 필터링(;, &&, ||, |)하도록 웹 애플리케이션 소스 파일을 작성하고 웹방화벽 등 필터링이 가능한 보안 장비를 운영중이라면 룰셋을 적용한다, 웹 애플리케이션 운영상 운영체제 명령어를 사용해야 한다면 허용할 수 있는 명령어 리스트를 선정하여 해당 명령어만 실행할 수 있도록 한다 -> 화이트 리스트 선정

* 파일업로드 취약점 : 파일업로드 기능이 존재하는 웹 애플리케이션에서 업로드 파일에 대해 적절한 제어를 하지 않을 경우 허가되지 않은 파일들이 웹서버로 업로드될 수 있는 취약점 -> 업로드하는 파일타입이나 확장자를 체크하지 않고 업로드를 허용하는 경우 웹쉘이 업로드되어 서버에서 악의적인 명령이 수행되거나 악성코드 파일이 업로드되어 이를 다운받은 사용자 PC가 침해사고를 당할 수 있다. 업로드 하는 파일의 개수나 크기에 제한을 두지 않을 경우 공격자는 다량의 큰 파일을 연속적으로 전송하여 시스템 부하나 장애를 유발시킬 수 있다.

cf. 웹쉘 : 악의적인 목적으로 웹서버 내 임의의 명령을 실행할 수 있는 서버 사이드 스크립트 파일을 말하며 다양한 언어로 만들어짐

cf. Base-64 인코딩/디코딩 -> 초기 텍스트 형식만 지원하던 메일에서 binary file을 첨부하기 위해 만들어진 인코딩/디코딩 방식을 의미함. (binary file -> ascii 문자)

cf. MIME -> Multipurpose Internet Mail Extension의 약자로 이메일과 함께 파일을 텍스트 형식으로 변환하기 위해 만들어진 표준 포맷으로 현재는 웹을 통해 여러 형태의 파일을 전달하기 위한 목적으로 사용함 (타입/서브타입 형식으로 구성 됨)

* 파일업로드 취약점에 대응하기 위한 웹서버 설정 - htaccess
-> 파일 업로드 디렉터리에 액세스 파일(.htaccess)를 생성한 후 FileMatch 지시자를 이용하여 특정 확장자에 대한 URL 직접 호출을 차단할 수 있다.
-> AddType 지시자를 이용하여 서버 사이드 스크립트 확장자를 가진 파일들에 대해 MIME 타입을 재조정할 수 있다.

```
<FileMatch "₩.(phlincllib)">
        Order allow, deny
        Deny from all
</FileMatch>

AddType text/html .html .htm .php
```

* 업로드 파일에 대한 파일 크기 검증 - http.conf 파일의 Directory 섹션에서 LimitRequestBody 지시자를 통해 지정할 수 있음 -> HTTP 요청 메시지 바디부에 데이터를 담아서 전달할 수 있는 최대 크기를 의미함. 0으로 지정하면 무한대를 의미하며 지정한 크기를 초과하여 요청할 경우 413 에러 응답 메시지를 반환함

* 파일 업로드 취약점 대응책 : 업로드 파일에 대한 파일타입 및 확장자 검증 정책을 통해 악성 스크립트 파일 업로드를 차단한다. 업로드 파일을 저장하기 위한 전용 디렉터리를 별도로 생성하고 웹서버 설정을 통해 업로드 디렉터리에 있는 서버 사이드 스크립트 파일이 실행되지 않도록 설정하거나 직접 URL 호출을 차단하도록 설정한다 (-> 아파치 웹서버를 사용할 경우 http.conf에 해당 Directory 섹션의 AllowOverride 지시자에 FileInfo 또는 All을 추가한 후 파일 업로드 디렉터리에 액세스 파일 (.htaccess)를 생성한다.), 업로드 하는 파일의 개수나 크기에 적절한 제한을 설정한다

* 파일 다운로드 취약점 : 파일 다운로드 기능이 존재하는 웹 애플리케이션에서 파일 다운로드 시 파일의 경로 및 파일명을 파라미터로 받아 처리하는 경우 이를 적절히 필터링 하지 않으면 공격자가 이를 조작하여 허용되지 않은 파일을 다운받을 수 있는 취약점 -> 시스템 환경설정 파일, 소스 코드 파일, 데이터베이스 연동 파일 등 (중요파일) 을 다운 받을 수 있음

-> 파일 다운로드가 가능한 페이지의 URL/파라미터를 조작하여 임의의 파일 다운로드가 가능할 경우 취약한 것으로 판단함

* 파일 다운로드 취약점 : 허용하는 경로 이외의 디렉터리와 파일에 접근할 수 없도록 처리함, 사용자 입력값에 대해 파일 경로를 변경할 수 있는 문자열(../ 이나 ..₩ 와 같은 것들)을 필터링하도록 웹 애플리케이션 소스 파일을 작성하고 웹방화벽 등 필터링이 가능한 보안장비를 운영 중이라면 필터링 룰셋을 적용한다.

* 경로 추적 취약점 (Path Traversal, 경로 순회 취약점) : 웹서버와 웹 애플리케이션 파일 또는 디렉터리 접근이 통제되지 않아 웹서버 또는 웹 애플리케이션의 중요한 파일과 데이터에 대한 접근 및 실행이 허용되는 취약점을 말한다 -> 웹 애플리케이션의 루트 디렉터리를 벗어나 시스템의 주요 파일과 데이터에 접근할 수 있는 취약점 -> 상대 경로 참조 방식을 사용하여 경로를 조작함
-> 요청 파라미터값을 임의의 경로가 포함된 값으로 변경하여 웹 루트 디렉터리보다 상위 디렉터리에 있는 파일에 접근할 수 있으면 취약한 것으로 판단함

cf. 주요 경로 조작 문자 및 인코딩 방식
- .(dot) : .(%2e)
- /(slash) : /(%2f)
- ₩(black-slash) : ₩(%5c)

* 경로 추적 취약점 대응책 : 사용자가 임의로 접근할 수 있는 최상위 디렉터리를 웹 루트 디렉터리로 설정하여 웹 서버의 시스템 루트 디렉터리로 접근하지 못하도록 제한한다. -> 유닉스/리눅스에서의 chroot 기능, 윈도우의 새로운 논리 드라이브 생성 , 사용자 입력값에 대해 경로를 조작할 수 있는 문자를 필터링하도록 웹 애플리케이션 소스 파일을 작성하고 웹방화벽 등 필터링이 가능한 보안장비를 운영중이라면 필터링 룰셋을 적용한다.

cf. chroot : 유닉스/리눅스 시스템에서 chroot는 Change Root Directory 의 줄임말로 임시 루트 디렉터리를 설정하는 기능을 말함. 시스템 루트 디렉터리인 /가 아닌 임의의 디렉터리를 루트 디렉터리인것처럼 설정하여 불필요한 루트 디렉터리 접근을 차단할 수 있음

* 파일 삽입(File Inclusion) 취약점 : 공격자가 악성 서버 스크립트를 서버에 전달하여 해당 페이지를 통해 악성코드가 실행되도록 하는 취약점 -> 삽입할 악성 서버 스크립트 파일의 위치가 로컬 서버에 위치하는지 원격지에 위치하는지에 따라 LFI(Local File Inclusion)과 RFI(Remote File Inclusion)으로 나뉘어짐

* 파일 삽입 취약점에서 php.ini 파일 -> allow_url_fopen = Off 설정을 통하여 악성 스크립트 파일이 URL 형태로 include 되지 않도록 한다. 또한 에러 페이지도 출력하지 않는 것이 권고 되므로 display_errors=Off 도 해주는게 좋다

* 파일 삽입 취약점 대응책 : 소스코드에 include, require 등의 구문/함수가 존재하는지 검증하고 만약 사용자의 입력값을 통해 파일명이 결정된다면 외부의 악의적인 파일이 포함되어 실행되지 않도록 PHP 설정 파일인 php.ini 파일에서 allow_url_fopen을 Off 설정한다.

* URL/파라미터 변조 취약점 : 실행경로에 대해서 접근제어를 검사하지 않거나 불완전하게 구현하여 공격자가 URL/파라미터 값을 조작하여 중요정보에 접근 가능해지는 취약점 -> 사용자 입력값에 대한 검증 누락이 발생하는 모든 상황을 포함하고 있으므로 SQL인젝션, 불충분한 인증 및 인가, 크로스사이트스크립트 공격에 활용될 수 있음
-> GET/POST 방식의 파라미터 전달 방식은 웹 프록시 등의 도구를 통해 손쉽게 노출 및 변조를 할 수 있으므로 입력값에 대한 적절한 검증 절차, 서버 세션의 활용등을 고려하는게 좋다
-> URL/파라미터를 조작하여 일시적인 권한 상승 또는 정보 유출 등이 발생할 경우 취약한 것으로 판단함

* URL/파라미터 변조 대응책 : 파라미터 변조 여부에 대해 검증 과정이 필요하며 중요한 정보가 있는 페이지는 재인증을 적용한다. 웹 애플리케이션이 제공하는 정보와 기능을 역할에 따라 배분함으로써 사용자가 변경할 수 있는 데이터의 노출을 최소화 시킨다.

* 불충분한 세션 관리 취약점 : 웹 애플리케이션에서 사용자가 로그인할 때 매번 동일한 세션 ID를 발급하거나 세션 타임아웃을 너무 길게 설정하였을 경우 공격자가 다른 사용자의 세션을 재사용하여 해당 사용자의 권한을 탈취할 수 있는 취약점을 의미한다
-> HTTP 세션 하이재킹등이 이용 됨
-> 세션 ID를 분석 및 변조하여 로그인 시도 시 로그인에 성공할 경우 해당 취약점이 존재하는 것으로 판단한다

* 불충분한 세션 관리 취약점 대응책 : 세션 ID는 로그인시마다 추측할 수 없는 랜덤함 값으로 새롭게 발급한다, 세션 타임아웃 설정을 통해 일정시간 요청이 없으면 자동로그아웃이 되도록 구현한다.

-> php.ini 에서 session.gc_maxfiletime 을 통해 초단위 세션 유지시간을 지정할 수 있다
-> httponly 속성 쿠키를 이용하면 http 또는 https 를 이용한 전송에서만 쿠키를 사용할 수 있으므로 클라이언트 스크립트를 통한 참조가 불가능해지기 때문에 XSS 공격을 방지할 수 있다. php.ini 의 session.cookie_httponly = 1 or True
-> secure 속성 쿠키를 이용하면 https 통신일 경우에만 해당 쿠키를 전송하는 옵션이므로 전송구간 암호화를 통해 평문 쿠키가 전송 과정에서 노출되는 것을 방지할 수 있음 -> session.cookie_secure = 1 or Ture 설정을 통해 쿠키의 Secure 속성 활성화 가능

* 정보 누출 취약점 : 민감한 정보가 개발자의 부주의로 인해 노출되는 것으로 중요 정보를 주석 구문에 포함해 의도하지 않게 정보가 노출되거나 에러 페이지 또는 에러 메시지를 통해 불필요한 정보가 노출되는 취약점
-> 자체만으로 피해가 크지는 않지만 공격의 실마리를 제공해주는 역할을 함
-> 디폴트 에러 페이지 또는 지정된 에러메시지에 의한 웹서버 정보 노출이 이에 해당 됨
-> 웹서버 설정파일 http.conf 를 통해 사용자 정의 에러페이지를 설정할 수 있음

-> 에러 페이지 설정 예시

ErrorDocument 403 /home/security/eror_page.php
ErrorDocument 404 /home/scurity/error_page.php

-> php 환경이라면 php.ini 파일의 display_errors = Off 설정을 통해 에러 메시지 출력을 방지할 수 있음

* 기타 웹 애플리케이션 취약점
- HTTP 응답 분할 (Response splitting) 취약점 : 클라이언트의 HTTP 요청에 포함된 요청 파라미터값이 HTTP 응답 헤더에 포함되어 클라이언트에게 다시 전달될 때, 파라미터에 개행문자인 CR 과 LF가 포함되어 있으면 HTTP 응답이 2개 이상으로 분리 될 수 있음 -> 이 경우 공격자가 개행 문자를 이용하여 첫 번째 응답을 종료시키고 두 번째 응답에 악의적인 코드를 삽입하여 XSS 공격을 수행할 수 있음 -> %0D, %0A 를 잘 확인 해야 함
-> 클라이언트 요청 파라미터(입력값)가 서버 프로그램에서 쿠키에 응답 헤더로 사용되거나 리다이렉션을 위해 Location 응답헤더로 사용되거나 , 기타 응답헤더 설정에 사용될 경우 HTTP 응답 분할이 발생하지 않도록 적절히 필터링하여 사용한다

*XXE(XML eXternal Entity) 인젝션 취약점 : 취약한 XML 파서가 외부개체를 참조하는 XML 데이터를 처리할 때 공격자가 삽입한 공격 구문이 외부 개체가 동작하여 서버 파일 접근, 불필요한 자원 사용, 인증 우회, 정보 노출 등이 발생할 수 있는 취약점

-> 웹 애플리케이션에서 사용하는 XML 파서의 외부 개체 참조 기능을 사용하지 않도록 비활성화 한다.
-> php 의 경우 libxml_dissable_entity_loader(true) 를 통해 외부 개체 사용을 비활성화 할 수 있음
-> XML 문서 내 외부개체를 사용하지 못하도록 외부 개체를 선언하는 DTD를 필터링

cf. XML 파서 : XML 문서의 평문 데이터를 읽어들여 문법 검사 및 프로그램 내에서 접근하고 조작할 수 있도록 XML DOM객체 등으로 변환해주는 도구를 의미함
cf. DTD : XML 문서에서 XML 객체(반복적으로 사용되는 문장이나 문자열 등의 데이터를 쉽게 참조할 수 있도록 정의해 놓은 것을 말하며 정의한 개체는 XML 문서 내에서 참조 됨)의 정의를 선언 하는 부분. XML 문서의 구조 및 Element, Attribute, Entity 를 정의하는 목적으로 사용함. 여기서 XML 객체는 내부에서 선언한 내부객체와 외부자원에서 가져와 정의한 외부개체로 나뉘는데, 외부 객체는 SYSTEM 키워드를 사용하여 정의 한다

* XPath/XQuery 인젝션 취약점 : 데이터베이스와 연동된 웹 애플리케이션에서 XPath 및 XQuery 질의문에 대한 필터링이 제대로 이루어지지 않으면 공격자가 입력이 가능한 폼에 조작된 질의문을 삽입하여 인증 우회를 통해 인가되지 않은 데이터를 열람할 수 있는 취약점을 말한다
-> XPath, Xquery에 사용되는 외부 입력데이터에 대하여 특수문자 및 질의 예약어 필터링을 수행한다. 또한 스트링을 연결하는 형태로 구성하지 않고 인자(파라미터)화된 질의문을 사용한다

* 악성 컨텐츠 취약점 : 웹 애플리케이션에서 사용자 입력값에 대한 필터링이 제대로 이루어지지 않으면 공격자가 악성 컨텐츠를 삽입할 수 있으며 악성 컨텐츠가 삽입된 페이지에 접속한 사용자는 악성코드 유포 사이트가 자동으로 호출되어 악성 코드에 감염될 수 있는 취약점을 말한다. -> 악성 컨텐츠는 SQL Injection, Cross Site Script, 파일 업로드를 통한 페이지 위변조 기법 등을 통해 삽입할 수 있으므로 해당 취약점을 반드시 제거하여 악성 컨텐츠를 삽입할 수 없도록 해야 함

* 불충분한 인증 및 인가 취약점 : 웹 애플리케이션에서 개인정보 수정 페이지나 통합 로그인과 같은 곳에서 사용자 인증이 미흡할 경우 공격자가 파라미터로 전달되는 값을 수정하여 사용자 도용 및 개인정보 노출 문제가 발생할 수 있는 취약점을 말함 -> 공격자가 파라미터 변조를 통해 권한 없는 사용자의 인증/인가를 요청할 경우 로그인 한 사용자들의 아이디 정보를 세션에 저장하여 세션에 저장된 아이디 값과 전달된 파라미터 값을 비교하여 사용자 인증/인가를 할 수 있도록 함, 근본적으로 파라미터로 값을 전달하는 것보다는 세션을 통하여 변수를 저장하고 관리하는 것이 안전함

cf. 인증(Authentication) : 자신의 신원을 알리고 정말로 신원이 올바른지 증명하는 과정 / 인가 : 인증에 성공한 사용자를 대상으로 자원에 대한 접근 또는 사용을 허용할지를 결정하는 과정 -> 서비스 관리 측면을 의미함

* 취약한 패스워드 복구 취약점 : 웹 애플리케이션에서 존재하는 비밀번호 찾기 기능 또는 관리자에 의한 임시 비밀번호 발급 시 사용자 인증이 미흡하거나 비밀번호를 화면에 즉시 출력할 경우 공격자가 불법적으로 다른 사용자의 비밀번호를 획득, 변경, 복구할 수 있는 취약점을 의미함 -> 사용자를 식별하기 위한 수단 활용시 그 사용자의 유일한 값을 사용하고 사용자 본인인증 매커니즘 구현시 추측이 불가능하고 임시 패스워드는 안전한 난수 값을 사용한다

* 자동화 공격 취약점 : 특정 프로세스에 대한 접근시도 횟수 제한을 설정하지 않으면 공격자가 자동화 도구 및 봇을 활용하여 일 분에 수백번의 접근을 시도할 수 있으며 특정 프로세스를 반복 수행함으로써 자동으로 수많은 프로세스가 진행되어 시스템 성능에 영향을 미칠 수 있는 취약점 -> 일정 횟수 이상 로그인 시도가 발생하면 자동화 공격으로 인식하여 로그인할 수 없도록 차단함, 서버에 요청하는 사용자가 실제 사람인지 컴퓨터 프로그램인지 구별할 수 있도록 회원가입 및 로그인 등의 페이지에 캡차를 적용함 (CAPTCHA)

cf. CAPCHA (캡차) : 실제 사람인지, 컴퓨터 프로그램인지를 구별하기위한 방법으로 사람이 쉽게 인식할 수 있는 임의의 문자와 숫자를 그림으로 변환하여 제시하고 제시된 문자는 왜곡되거나 덧칠되어 기계가 자동으로 읽기 어려운 상태로 만든다.

* 데이터 평문 전송 취약점 : 로그인 또는 실명인증시 민감한 데이터가 평문으로 통신채널을 통해 송수신될 경우 공격자가 감청을 통해 다른 사용자의 민감한 데이터를 획득할 수 있는 취약점 -> 중요한 데이터에 대해 송신 시 암호화를 적용하여 중간에 감청하더라도 그 내용을 알 수 없게함 , HTTPS 과 같은 암호화 통신 사용

* 쿠키 변조 : 쿠키를 공격자가 변조하여 다른 사용자로 전환하거나 권한 상승이 가능한 취약점 -> 웹 애플리케이션 운영 시 가급적 쿠키 방식 대신 세션 방식을 사용하고 부득이하게 쿠키를 사용해야 할 경우 암호화를 적용 함

* 개발 보안 관리 : 애플리케이션 서버의 환경 설정에 대해 보안 강화 가이드라인을 만들어 이를 준수하도록 해야 함
-> 사용자에게 전달된 값을 재사용할 경우 신뢰 X
-> 최종 통제 매커니즘은 반드시 서버에서 수행
-> 클라이언트에 중요 정보를 전달하지 않음
-> 중요정보 전송시 POST 메소드 및 SSL을 적용함
-> 중요한 트랙잭션이 일어나는 프로세스에 사용자의 비밀번호를 재확인
-> 중요정보를 보여주는 페이지는 캐시를 사용하지 못하도록 설정함 : HTML HEAD 부분에 CONTENT="no-cache" 추가
-> 적절한 방법으로 암호화 : 공인된 암호화 알고리즘 사용 필요 (자체 개발 암호화 알고리즘 사용 X), 암호화키를 사용하지 않는 알고리즘은 암호화 알고리즘이 아니라 단순 인코딩 알고리즘으로 기밀성 보장이 안됨
-> 각 언어에서 제공하는 보안 수단을 이해한 후 사용하자

* 디렉터리 리스팅 취약점
-> 웹 서버의 미흡한 설정으로 인해 인덱싱(리스팅) 기능이 활성화가 되어 있으면 공격자가 강제 브라우징을 통해 서버 내의 모든 디렉터리 및 파일 목록을 볼 수 있는 취약점
-> 웹서버 구조, 백업 파일이나 소스파일, 미공개 파일등이 노출 될 수 있음

* 디렉터리 리스팅 취약점 대응책 : apache 기준
- httpd.conf 를 통해 디렉터리 인덱싱 기능을 제거한다

<Directory />
Options Indexes -> Indexes 로 되어 있으면 인덱싱 기능이 활성화 되어 모든 디렉터리의 파일 목록이 노출되므로 Indexes 대신에 None으로 변경해야 함
</Directory>

-> 윈도우의 IIS 웹서버의 경우 '디렉터리 검색' 기능을 해제 해서 리스팅을 제거 할 수있음
-> tomcat 웹 서버의 경우 web.xml 파일의 default 서블릿 초기 파라미터 설정에 listings를 false로 설정함

* 웹 서비스 메소드 설정 취약점 : PUT, DELETE 등의 불필요한 메소드를 허용하였을 경우 공격자가 이를 이용하여 웹서버에 악성 파일을 생성하거나 중요 파일의 삭제 및 수정 등이 가능한 취약점

-> DocumentRoot 디렉터리 섹션에 LimitExcept 지시자를 이용하여 불필요한 메소드 사용 제한을 할 수 있음

<LimitExcept GET POST>
Order allow,deny
Deny from all
</LimitExcept>

* 관리자 페이지 노출 취약점 : 웹 애플리케이션의 전반적인 기능 설정 및 회원 관리를 할 수 있는 관리자 페이지가 추측할 수 있는 형태로 구성되어 있으면 공각자가 관리자 페이지에 쉽게 접근할 수 있다 -> 무작위 대입 공격을 통하여 관리자 권한을 획득할 수 있는 취약점이 생김 -> 관리자 페이지는 가장 보호 되어야할 페이지로 매우 제한적으로 접근을 허용해야 함
-> 관리자 페이지의 URL을 쉽게 추측하기 어렵게 만들고 관리자 페이지에 대한 접근 제어를 설정해준다

cf. apache 웹서버 httpd.conf 설정에서 Order 지시자는 Allow, Deny 순서를 명시하는 것으로 뒤에오는 설정이 우선된다 -> Order Deny,Allow 로 설정되어 있으면 Allow 설정이 우선시 된다

* 위치 공개 취약점 : 개발과정 또는 소스코드를 수정하였을 경우 백업파일, 로그파일, 압축파일과 같은 파일이 자동적으로 생성되어 웹 애플리케이션 상에 노출될 경우 공격자가 유추 후 직접 접근을 요청하여 핵심 정보를 획득할 수 있는 취약점을 의미함 -> 웹서버에 테스트 파일과 같은 불필요한 파일을 삭제하고 서비스와 관련 없는 디렉터리는 일반 사용자의 접근이 불가능하도록 적절한 접근 제어를 수행한다.

* 검색엔진 정보 노출 취약점 : 검색엔진에 의해 웹 사이트 해킹에 필요한 정보가 검색되어 해킹의 빌미가 제공되는 취약점을 말함 -> '로봇배제표준'은 검색로봇에 대한 웹 사이트의 디렉터리 및 파일들에 대한 검색조건을 명시해놓은 국제 규약으로 접근 제한에 대한 설정을 'robots.txt'(ASCII 또는 UTF-8) 파일에 기술한다.
-> 로봇 배제표준은 방지 기술이 아닌 검색로봇 운영자 간의 약속이므로 악의적인 검색엔진의 경우 이를 무시하고 컨텐츠를 수집할 수 있음.
-> 브라우저 주소창에 URL/robots.txt 형식으로 입력시 해당 파일이 없는 경우 검색 엔진 접글 차단 설정이 미적용된 상태

cf. 검색 로봇 : 검색 엔진이 검색 데이터베이스의 내용을 보충하기 위해 웹 사이트를 검색하여 정보를 가져오는 프로그램. 일반적인 검색로봇은 사이트 방문시 가장 먼저 웹 사이트의 로봇배제표준 설정파일 (= robots.txt) 을 확인하고 이를 준수하여 컨텐츠를 수집함. 구글의 googlebot, 네이버의 naverbot, yeti 등이 있음
-> robots.txt 파일은 반드시 웹 사이트의 최상위 주소에 저장해야 함. 하위 디렉터리에 저장할 경우 효력이 없다

* robots.txt 작성시 유의사항 : 설정파일명은 반드시 robots.txt 이어야 하며 파일 위치는 웹 사이트의 최상위 디렉터리여야 한다. 대소문자를 구분한다.(설정에 있는 User-agent, Disallow 등의 대소문자를 반드시 구분하도록 하자) 띄어 쓰기에 유의해야 한다. 콜론은 붙어 있어야 하며 콜론 뒤에는 공백이 추가 된다. 다중 검색로봇 지정시 한 줄을 띄워야 한다.

-> 모든 검색로봇에 대해 웹 사이트 전체에 대한 접근을 차단함
User-agent: *
Disallow: /

-> Googlebot, Yeti 검색로봇에 대해 웹 사이트 전체에 대한 접근 허용을 허용한다. Disallow 항목에 아무것도 명시하지 않으면 웹 사이트 전체 허용을 의미하며 'Allow: /'와 동일한 의미를 가짐
User-agent: Googleboot
User-agent: Yeti
Disallow:

-> 모든 검색로봇에 대해 cgi-bin, private 디렉터리에 대한 접근을 차단함 (디렉터리 참조시 반드시 '/'로 종료되어야 함)
User-agent: *

Disallow: /cgi-bin/
Disallow: /private/

-> 특정 파일의 확장자를 차단하는 예제
User-agent: Google-image
Disallow: /*.pdf$

* 기타 웹서버 보안 대책(아파치 기준)
  - 웹 서비스 최소 권한 사용자 운영 : 웹서버의 보안 취약점으로 인한 권한 탈취가 발생할 가능성이 항상 있기 때문에 웹서버 운영 시 위험을 최소화하기 위해서 최소한의 권한을 가진 사용자와 그룹으로 웹서버 프로세스를 실행 시켜야 함
-> 웹 서버 프로세스를 실행할 별도의 사용자 및 그룹을 생성하고 해당 사용자로 시스템에 로그인할 수 없도록 설정함
-> 별도로 생성한 사용자의 로그인 쉘 부분을 /sbin/nologin 또는 /bin/false로 지정하여 시스템에 로그인할 수 없도록 한다
-> 아파치는 80/tcp 포트를 열고 클라이언트 연결 요청을 처리하는 root 권한의 리스닝 프로세스와 다수의 서비스 요청을 처리하는 apache 권한의 서버 프로세스로 구성 됨

- 심볼릭링크 사용 설정 제거 : 만약 시스템 자체의 루트 디렉터리에 심볼릭 링크를 걸게 되면 웹서버 구동시 사용자 권한으로 모든 파일 시스템의 파일에 접근할 수 있게 되어 passwd 파일과 같은 민감한 파일을 누구나 열람할 수 있는 문제점이 발생함
-> <Directory /> -> 모든 디렉터리 섹션을 의미
    Options FollowSymLinks -> 심볼릭링크를 허용하는 옵션임 None으로 바꿔주도록 하자 (Options None)
    AllowOverride All
  </Directory>

- 웹 서비스 정보 숨김 : 클라이언트와 웹서버가 통신할 경우 웹서버에서는 응답 헤더나 에러 메시지 등에 웹서버 버전이나 종류, OS 정보, 응용 프로그램 버전 등을 전송함 -> 공격자에 의해 웹서버 버전별 또는 동작하는 응용 프로그램에 잘 알려진 취약점을 공격하는 데 유용하게 이용되기 때문에 공격자에게 웹서버의 버전과 같은 배너 정보를 숨기는 것이 안전함

-> ServerTokens 를 Prod 로 써서 Server의 최소정보만 노출 하도록 한다 (Prod : 웹서버 종류, Min : 웹서버 종류 + 웹서버 버전, OS : 웹서버 종류 + 웹서버 버전 + 운영체제, Full : 웹서버 종류 + 웹서버 버전 + 운영체제 + 설치된 모듈 정보
-> ServerSignature 지시자에 Off 를 설정하여 불필요한 웹서버 관련 정보가 전송되지 않도록 한다

* 웹서버 설정 파일 주요 지시자
- ServerRoot : 웹 서버가 설치된 최상위 디렉터리 경로를 지정함 (설정 파일, 로그 파일, 모듈 등이 위치)
- Listen 80 : 리스닝 포트를 지정
- User nobody : 제한된 권한을 가진 nobody 또는 애플리케이션 사용자 그룹 지정을 권장. 80 포트를 사용하는 리스닝 프로세스는 root, 실제 서비스를 수행하는 서버 프로세스는 nobody(일반사용자)
- ServerTokens Prod : HTTP 응답 시 헤더의 Server 필드를 통해 제공할 웹서버, OS, 모듈 등의 정보 레벨을 지정
- ServerSignature Off : 웹서버에서 직접 생성하는 에러 메시지, 디렉터리 리스팅 메시지 등에 웹서버, OS, 모듈 등의 정보를 제공할지를 지정함
- ServerAdmin : 클라이언트에 전달되는 에러 메시지에 포함될 관리자 메일 주소를 설정함
- DirectoryIndex index.php index.html : 클라이언트가 디렉터리만 명시하여 요청한 경우 이를 처리할 자원 목록을 지정함
- Timeout 120 : 클라이언트의 요청으로 서버와 연결된 후 클라이언트와 서버 간에 아무런 메시지가 발생하지 않을 동안의 대기 시간으로 초 단위로 지정 됨
- KeepAlive On : 클라이언트와 서버 프로세스 간의 연결을 일정 조건에 따라 지속시켜 동일한 연결을 통해 동일한 클라이언트의 다수 요청을 처리하는 기능의 사용 여부를 지정함
- MaxKeepAliveRequests 100 : KeepAlive 기능 사용 시 클라이언트와 서버 프로세스 간에 연결을 지속시키는 동안에 허용할 최대 요청 건수를 지정함. 해당 요청 건수를 초과하면 연결을 종료함
- KeepAliveTimeout 15: KeepAlive 기능 사용시 클라이언트와 서버 프로세스 간에 연결을 지속 시키는 동안에 클라이언트의 마지막 요청 이후 다음 요청을 대기하는 시간을 지정함. 해당 시간 동안 클라이언트의 추가 요청이 발생하지 않으면 연결을 종료함
- RequestReadTimeout header=5 body=10 : 클라이언트로부터 요청 메시지의 헤더부와 바디부를 모두 수신하는 시간에 대한 타임아웃을 지정함. 지정한 시간동안에 요청 메시지의 헤더부와 바디부를 모두 수신하지 못한 경우 408 응답 메시지를 반환함. Slow 계열의 HTTP 도스 공격에 효과적으로 대응할 수 있음
- ErrorLog logs/error_log : 에러 로그파일의 경로를 지정함 -> 상대 경로를 사용할 경우 설치 최상위 디렉터리를 기준으로함
- CustomLog logs/access_log combined : 접속 기록을 저장하기 위한 로그 경로를 지정함

* 웹 로그 분석 : 클라이언트의 요청으로 웹서버가 응답한 내용은 액세스 로그에 기록, 클라이언트 요청으로 웹서버에 오류가 발생했을 경우 에러로그에 기록 됨 -> access log, error log를 웹 로그라고 함

-> 접속 사용자의 행위/취향 등을 분석
-> 보안사고 발생 시 추적할 수 있는 증거자료

-> 보안사고 발생 전 이상 징후, 해킹시도, 해킹 성공 여부를 확인
-> 다양한 웹 공격 패턴 파악
-> 접속 시간, 접근 파일 정보, 사용자 정보, 요청 방식, 성공 여부 등을 확인

* 웹 로그 종류 : NCSA CLF(Common Log Format) -> Apache, Tomcat / NCSA ELF(Extended Log Format) -> Referer 값과 User-Agent 가 추가된 포맷 / W3C ELF -> 주로 IIS 에서 사용하며 Referer 과 User-Agent 를 비롯한 Cookie 및 사용자 정보를 추가로 남기리 수 있음

* NCSA ELF 포맷
- Host : 클라이언트의 호스트 명 또는 IP주소
- Ident : 클라이언트의 사용자 이름, 보통 '-'로 대체
- Authuser : 인증이 요청된 원격 사용자 이름, 보통 '-'로 대체
- Date and Time : 요청을 보내온 시간과 날짜에 대한 정보
- Request : Client 가 보내온 요청라인에 대한 정보
- Status : Client 요청에 대한 처리 상태를 나타내는 상태코드
- Bytes : 서버에서 클라이언트로 전송된 응답데이터의 크기(헤더 제외)
- Referer : 요청된 URL이 참조되거나 링크된 URL
- User-Agent : 클라이언트 OS 및 브라우저 버전 정보

* 제로보드 공격 : cmd 파라미터로 원격에서 임의 명령을 실행할 수 있는 취약점을 이용-> 제로보드는 php로 개발한 웹 게시판 시스템을 의미함
* 테크노트 공격 : main.cgi 의 filename파라미터로 원격에서 임의 명령을 실행할 수 있는 취약점을 이용

* 보안 서버 구축 : 인터넷상에서 정보를 암호화 하여 송수신하는 기능이 구축된 웹서버를 의미함 -> 독립적인 하드웨어를 따로 설치하는 것이 아니라 이미 사용하고 있는 웹서버에 SSL/TLS 인증서나 암호화 소프트웨어를 설치하여 암호 통신을 지원함
-> 정보 유출 방지
-> 위조/가짜 사이트 방지 (피싱방지)
-> 기업 신뢰도 향상

* 보안서버 구축의 법적근거
- 개인정보법 관련 개인정보보호위원회 고시 개인정보의 기술적/관리적 보호조치 기준 : 보안서버라 함은 정보통신망에서 송,수신하는 정보를 암호화하여 전송하는 웹서버를 의미한다, 정보통신 제공자등은 정보통신망을 통해 이용자의 개인정보 및 인증정보를 송,수신할 때에는 안전한 보안서버 구축 등의 조치를 통해 이를 암호화 해야 한다. 보안서버는 다음 중 하나의 기능을 갖추어야 한다
-> 웹서버에 SSL 인증서를 설치하여 전송하는 정보를 암호화하여 송,수신하는 기능
-> 웹서버에 암호화 응용프로그램을 설치하여 전송하는 정보를 암호화하여 송,수신하는 기능

- 개인정보보호법 관련 개인정보보호위원회 고시 개인정보의 안전성 확보 조치 기준 : 개인정보처리자는 고유식별정보(주민등록번호, 여권번호, 운전면허번호, 외국인등록번호), 비밀번호, 생체인식정보를 정보통신망을 통하여 송신할 경우 이를 암호화 하여야 한다

* SSL/TLS 서버 인증서 생성 시 보안 가이드 : 키쌍 생성시 2048-bit 이상의 키를 사용하고 서버 개인키에 대한 접근은 최소화 하며 안전하게 관리한다, 인증서 발급시 인증서 서명 알고리즘을 SHA-2(256-bit)이상으로 설정한다, 인증서 발급시 주체/소유자의 CommonName을 서브 도메인까지 정확히 명시한다, 신뢰할 수 있는 인증기관을 통해 인증서를 발급한다

* Open SSL 라이브러리 보안가이드 : 취약점이 발견된 OpenSSL 라이브러리 버전을 사용할 경우 보안 업데이트를 수행한다.

* SSL/TLS 웹서버 설정 보안 가이드 : 취약한 암호화 통신 방법으로 알려진 SSL2.0, SSL3.0 프로토콜을 비활성화 하고 TLS1.0 ~ TLS1.2 프로토콜을 사용함, 취약한 암호방식(키교환 알고리즘, 암호 알고리즘, 해시 알고리즘) 을 사용하는 암호도구목록(Cipher Suite)을 제거함
-> SSL/TLS 협상 과정에서 키 교환 시 익명 디피-헬만을 사용하면 서버 DH 파라미터 교환 과정에서 인증하지 않는 취약점이 있으므로 제거해야 한다. 임시 DH 의 경우 키 교환 과정에서 서버 DH 파라미터를 RSA, DSA 등의 서명 알고리즘을 통해 인증하므로 사용을 권장한다
-> SSLProtocol(서버에서 지원할 SSL/TLS 버전을 지정하기 위한 지시자), SSLCipherSuite(암호도구목록을 지정하기 위한 지시자) 를 통해 설정할 수 있다

* 메일 서버 구조
- MUA : Mail User Agent -> 사용자가 메일을 송수신하기 위해 사용하는 메일 클라이언트 프로그램으로 메일을 작성하거나 메일함에 도착한 메일을 확인하는 기능을 수행함, 아웃룩/선더버드 등이 있음
- MTA : Mail Transfer Agent -> 메일 서버 프로그램으로 수신한 메일을 분석하여 수신자가 자신이 아닌 메일주소라면 해당 주소의 메일 서버로 전송하고, 수신자가 자신의 메일 주소라면 MDA를 통해 각 사용자 메일함에 저장하도록 한다. 센드메일/큐메일 등이 있다

- MDA : Mail Delivery Agent -> 사용자 메일함에 메일을 저장해주는 프로그램으로 메일이 최종 수신 메일 서버에 도착했을 때 메일 서버는 MDA 에 메일을 전달하고 MDA는 해당 사용자 메일함에 메일을 저장한다, 프록메일등이 있다.
- MRA : Mail Retrieval Agent -> 메일 클라이언트가 확인을 요청하는 메일을 사용자 메일함에서 사용자로 전달해주는 프로그램으로 MUA와 MRA간 통신은 POP3 또는 IMAP 프로토콜을 기반으로 한다

*SMTP(Simple Mail Transfer Protocol) : 메일 전송 프로토콜 -> MUA와 MTA 또는 송수신 메일서버(MTA) 간에 메일 전송을 위해 사용하는 프로토콜로 TCP25번 포트를 사용한다. SMTPS(SMTP over SSL/TLS) 는 SMTP 통신 구간에 SSL/TLS 보안을 적용한 프로토콜로 TCP 465번 포트를 사용한다.

* POP3 : 메일 접근 프로토콜 -> 메일 클라이언트인 MUA 에서 메일 서버 MTA 로 부터 메일을 수신할 수 있또록 해주는 프로토콜로 TCP/110을 사용한다. 클라이언트가 메일을 서버로부터 다운로드 -> 서버에서 해당 메일을 삭제 하는 형태로 동작하기 때문에 다른 클라이언트에서 메일 확인이 불가능함, POP3S는 POP3통신 구간에 SSL/TLS 보안을 적용한 프로토콜로 TCP/995 사용

* IMAP : 메일 접근 프로토콜 -> 메일 클라이언트 MUA 에서 메일 서버 MTA 로 부터 메일을 수신할 수 있도록 해주는 프로토콜로 TCP/143 포트를 사용함. 메일을 다운로드가 아니라 복사해오는 방식이기 때문에 클라이언트가 메일을 서버로부터 가져온 후에도 서버에 해당 메일이 계속 남아있게 된다. 따라서 다른 클라이언트에서도 메일확인이 가능하다, IMAPS 은 IMAP 통신 구간에서 SSL/TLS 보안을 적용한 프로토콜로 TCP/993 을 사용함

* 이메일 시스템 동작 방식 : 사용자가 메일 클라이언트인 MUA 를 이용해 메일을 작성한 후 SMTP 프로토콜을 통해 클라이언트가 설정한 MTA 메일서버로 전송함 -> 메일서버인 MTA는 메일큐에 수신한 메일을 임시 저장한 후, 수신한 메일을 분석하여 수신자가 자신 아닌 메일주소라면 해당 주소의 메일 서버로 SMTP 프로토콜을 통해 전송하고 수신자가 자신의 메일주소라면 MDA 를 통해 각 사용자 메일함에 파일 형태로 저장하도록 함
-> sendmail 서버의 경우 메일 큐로 /var/spool/mqueue 디렉터리를 사용하며 각 사용자 메일함은 /var/spool/mail 디렉터리에 위치함
-> 수신자 메일주소의 메일 서버 확인 방법 : 메일 주소에 있는 도메인을 이용하여 MX 레코드에 대한 DNS 질의를 수행함
-> 사용자가 메일 클라이언트 MUA 를 통해 자신의 메일 확인을 요청하면 MRA는 해당 사용자의 메일함에 있는 메일을 POP3 또는 IMAP 프로토콜을 통해 클라이언트에 전달 함

* SMTP 메일 형식 -> SMTP는 텍스트 기반의 프로토콜로 봉투(메일 서버 간에 메일을 주고받기 위한 메일 주소 정보를 담고 있음. 메일 수신자에게 전달되는 정보가 아님, MAIL FROM, RCPT TO)와 메시지(메일 수신자에게 보이는 메일 내용으로 다양한 메시지 헤더와 메시지 바디로 구성되어 있음. 주요 헤더 정보는 From, To, Subject) 정보를 주고 받음,

* SMTP 프로토콜 취약점
- 발신메일 서버의 조작 -> SMTP 서버를 조작함 , SMTP Spoofing : 메일 봉투의 MAIL FROM 정보를 조작하여 실제와 다른 신뢰할만한 발신자 메일주소로 조작하여 수신 메일 서버를 속일 수 있음
-> SPF 메일 인증기술을 사용하여 MAIL FROM 에 명시한 발신자 메일주소가 정당한(인증된) 메일 주소로 부터 온 메일인지 검사 할 수 있음

- 메일 메시지 훔쳐보기 -> 스니핑 : 기본적으로 평문 텍스트로 메일을 전송하기 때문에 전송구간에서 공격자가 부정한 방법으로 메일을 가로채 메시지를 훔쳐보거나 조작하는 중간자 공격이 발생할 수 있음 -> Man In The Middle
-> 발신자 측에서 메일의 메시지에 대한 디지털서명(전자서명)을 추가하여 메일의 위변조 여부를 검사할 수 있는 DKIM(DomainKeys Identified Mail) 메일 인증 기술을 적용함 , 메일 서버와 클라이언트 통신 구간 보안 적용이 가능한 SMTPS, POP3S, IMAPS 등의 프로토콜을 설정하여 메일 스니핑 공격을 방지함

- DMARC(Domain-based Message Authentication, Reporting and Conformance) 메일 인증 기술의 적용 -> SPF 와 DKIM 인증 기술을 모두 적용하 수 있고 인증 결과에 대한 모니터링이 가능한 기술로 메일 보안을 한층 더 강화할 수 있음

* 메일서버 보안 설정 : SMTP 릴레이 -> 메일서버 외부에서 메일서버를 경유하여 다른 메일 서버로 메일을 발송하는 것을 말함. 이때 경유한 메일서버를 SMTP 릴레이 서버라고 표현함 ->SMTP 릴레이 서버의 발신자 인증 여부에 따라 오픈 릴레와 인증된 릴레이 서버로 구분할 수 있음. 오픈 릴레이 서버는 발신자에 대한 IP나 메일 계정에 대한 인증 없이 모든 메일을 릴레이 해주는 메일 서버로 인증을 수행하지 않기 때문에 스팸 메일 발신자의 메일서버로 악용되는 사례가 자주 발생한다. -> 허가된 IP 또는 IP대역, 도메인 등에서만 메일 릴레이가 가능하도록 발신자를 제한하고 STMP AUTH 기능을 사용하여 발신자 계정 인증을 수행한다

* 센드메일 서버 스팸메일 및 릴레이 차단 설정
- /usr/sbin/sendmail : sendmail 주 데몬 실행파일
- /usr/sbin/makeup : sendmail 맵 생성 실행 파일 (access, virtusertable 등의 db 파일 생성)
- /var/spool/mqueue : 메일 큐 디렉터리 -> 수신한 메일을 임시 저장하는 디렉터리
- /var/spool/mail : 개별 사용자 메일함 -> 개별 사용자별로 수신된 메일을 보관하는 디렉터리
- /etc/mail/access : 스팸메일 및 릴레이 차단을 위한 설정 파일 -> access 파일은 설정을 위한 텍스트파일로 실제 정책을 적용하기 위해서는 makeup 명령을 이용하여 access.db 파일을 생성해야 함
- /etc/mail/sendmail.cf : sendmail 설정 파일

cf. 센드메일 : 인터넷 전자 메일의 표준 규약인 STMP프로토콜을 통해서 메일 서버 간에 메일을 주고 받는 역할을 하는 무료 오픈소스 소프트웨어 . access DB라는 데이터베이스를 이용해 특정 메일의 차단 및 릴레이 제한을 설정할 수 있음

* 센드메일 access 파일 설정
- 적용대상지정(IP, IP대역, 도메인, 메일주소), 처리방식지정(OK, RELAY, REJECT, DISCARD)
- OK -> 지정한 대상의 메일을 조건 없이 모두 허용
- RELAY -> 지정한 대상의 메일 릴레이를 허용
- REJECT -> 지정한 대상의 메일을 모두 거부함 (거부 메시지 보냄)
- DISCARD -> 지정한 대상의 메일을 모두 폐기 (폐기 메시지 x) , 폐기 메시지가 반환되지 않기 때문에 실제로는 메일이 폐기 되지만 발신자 입장에서는 정상적으로 처리된 것으로 인식하게 됨
- 501 message -> 지정한 메일주소와 일치할 경우 메일을 거부하고 설정한 거부 메시지를 보냄

=> access 파일을 적용하기 위해서는 makemap 명령어를 사용하여 access.db 데이터베이스 파일을 생성해야함 ->
makemap hash /etc/mail/access < /etc/mail/access 또는
makemap hash /etc/mail/access.db < /etc/mail/access

* 이메일 인증 기술 -> 스팸 메일 방지 기술 SPF : Sender Policy Framework , 발신 메일서버 정보를 사전에 발신 도메인 DNS 서버에 공개적으로 등록함으로써 수신자가 수신한 메일의 발신자 IP주소가 발신자 메일 주소 도메인의 실제 메일서버 정보와 일치하는지 검사할 수 있도록 하는 이메일 인증 기술 = 발송자 메일서버 인증 또는 메일 서버 등록제라고 표현

* SPF 동작 방식
- 발신측 : SPF 레코드 정보(해당 도메인에서 사용하는 발신자 메일서버 정보와 수신측 처리 방식을 담고 있음)를 도메인의 TXT 레코드에 기록하여 등록함
- 수신측 : 수신자의 메일서버는 발신자 메일주소 도메인에 대한 SPF 레코드를 조회한 후 실제 메일을 발송한 메일서버의 IP가 조회한 SPF 레코드에 등록된 서버인지 여부와 처리 방식에 따라 수신 여부를 결정함

* SPF 레코드 형식 : 버전(spf 레코드 버전)과 하나 이상의 한정자(Qualifier, +,-,~,?)와 메커니즘(Mechanism, ip4, ip6, include, all)의 조합으로 이루어진 텍스트임. 여기서 메커니즘은 발신 메일 서버의 매칭 방식을 정의하여 순서대로 매칭 여부를 판단함

- 한정자의 종류 : + (인증 통과 -> 매커니즘과 일치할 경우 인증된 메일서버로 메일을 통과시키라는 의미. 메커니즘에 한정자를 지정하지 않은 경우 기본적으로 적용 됨), - (인증 실패 또는 심각한 인증 실패 -> 매커니즘과 일치할 경우 인증되지 않은 메일서버로 메일을 거부 하라는 의미. 일반적으로 수신 메일서버에서 스팸 메일함에 임시보관하거나 수신을 거부함), ~ (가벼운 인증 실패 -> 메커니즘과 일치할 경우 인증되지 않은 메일서버로 수신 측 메일서버 정책에 따라 허용 여부를 판단 함), ? (중립)

-> SPF 레코드 형식 예시
"v=spf1 ip4:211.252.150.8 ip4:202.30.50.0/24 all" -> 발신 메일서버의 IPv4 주소가 211.252.150.8 이거나 202.30.50.0/24 이라면 통과시키고 그 이외의 주소(all)라면 거부하라는 의미

"v=spf1 include:_SPFinclu.kisa.or.kr ~all" -> _SPFinclu.kisa.or.kr 도메인의 SPF 레코드 정보를 참조하여 메일 허용 여부를 판단하고 그 외의 주소라면 수신 측 메일 서버 정책에 따라 허용 여부를 판단하라는 의미임 (~, Soft Fail)

* DKIM : 발신자가 발신 메일의 메시지헤더와 바디에 대한 디지털 서명을 메일 헤더에 삽입하여 수신된 이메일이 위변조 되지 않았는지 수신자 측에서 검증할 수 있는 이메일 인증 기술 -> 도메인 키 인증메일

* DKIM 동작 방식
- 발신측 : 발신자 도메인의 DNS 서버에 수신 측에서 디지털 서명을 검증할 수 있는 정보를 담고 있는 DKIM 레코드 정보를 지정한 도메인의 TXT 레코드에 기록하여 등록함 -> 발신자 메일서버에서 발신 메일의 메시지 헤더와 바디를 개인키로 서명한 후 서명 값을 DKIM_Signature 에 추가하여 전송한다
- 수신측 : 수신자 메일 서버는 DKIM_Sginature 헤더에 있는 정보를 참조하여 공개된 지정한 domainkey 도메인에 대한 DKIM 레코드를 조회한 후 DKIM 레코드에 등록된 공개키와 서명 알고리즘을 이용하여 DKIM-Signautre 헤더에 있는 서명값 검증을 수행함

* DMARC(Domain-based Message Authentication, Reporting and Conformance) : SPF 인증 기술과 DKIM 인증 기술을 DMARK 인증 정책을 통해 모두 적용하여 정당한 발신자인지 여부와 메일의 위변조 여부를 검사하고 수신측에서 정책처리 결과에 대한 DMARC 보고서를 발신 측에 전달함으로써 발신 측에서 메일 모니터링을 할 수 있는 이메일 인증 기술 -> 도메인 기반 이메일 인증

- 발신측 : 발신자 도메인의 DNS 서버에 SPF 레코드와 DKIM 레코드를 우선적으로 등록하고 인증정책 정보를 담고 있는 DMARC 레코드 정보를 DMARC 용도의 도메인의 TXT 레코드에 기록하여 등록한다

- 수신측 : 수신자 메일서버는 SPF, DKIM 및 DMARC 레코드를 조회하고 DMARC 정책에 따라 인증을 수행한다. 인증에 실패한 메일에 대해서는 DMARC 보고서를 작성하여 발신 측에 전송한다

* 인증 기술 관련 주요 메일 헤더 정보
- Recevied 메시지 헤더 : 실제 메일이 전송된 경로를 나타냄. Recevied 헤더가 여러개인 경우는 여러 메일 서버를 경유했다는 의미이며 가장 아래 쪽에 보이는 헤더가 처음 메일을 발송한 메일서버이고 위로 올라갈수록 거쳐 온 메일 서버를 보여줌. Recevied 메시지 헤더는 메일서버를 경유할 때 마다 추가되는 헤더로 일반적으로 스팸 메일 분석시 발송 근원지를 역추적하기 위해 참고하는 중요한 서버이다

-> 형식 : Received: "from 발송 호스트" "by 수신호스트" "id 메일 고유 식별자" "for 수신자 메일주소" "수신 날짜/시간"

-> 통신 구간 보안을 위해 SSL/TLS를 지원하는 SMTPS를 적용한 경우에는 관련 정보를 확인할 수 있다.
(Received 헤더 TLS Cipher Suite 표기 예시 : 키교환알고리즘 인증/서명알고리즘-대칭 키 블록 암호 알고리즘- 블록 암호 운용모드-메시지 인증 알고리즘

- Recevied-SPF 메시지 헤더 : 수신자 메일서버에서 SPF 인증 결과를 기록하는 헤더임. SPF 인증 결과는 Authentication-Results 헤더에도 확인할 수 있음.

- Authentication-Results 메시지 헤더 : 수신 메일서버에서 SPF, DKIM 및 DMARC 인증 결과를 기록하는 헤더이다. 인증 기술에 따라 pass, fail 등의 값으로 기록 된다

* 이메일 인증(스팸 메일방지)를 위한 기술적 대응 방안 : 신뢰할 수 있는 발신자로 사칭하는 스팸 메일에 대응하기 위해서는 SPF (메일 서버등록제) 인증기술을 통해 인증된 발신자 메일서버로부터 온 메일인지 확인하고 DKIM(도메인 키 인증메일) 인증 기술을 통해 메일의 헤더나 내용이 위변조 되지 않았는지 확인하는 한편, DMARC (도메인 기반 이메일 인증) 인증 기술로 정책에 따른 SPF와 DKIM의 이중 적용 및 수신이 실패되는 메일에 대한 모니터링 등이 필요하다

* 데이터베이스 보안 위협
- 집성 (Aggregation) : 낮은 보안 등급의 정보 조각들을 조합하여 높은 보안 등급의 정보를 알아내는 것으로 개별 데이터 항목보다 종합 정보의 보안 등급이 높은 경우 기밀 정보 유출의 문제가 발생함
- 추론 (Inference) : 보안 등급이 없는 일반 사용자가 보안으로 분류되지 않은 정보를 정당하게 접근하여 기밀 정보를 유출해내는 행위 -> 주로 통계로 부터 발생하기 때문에 통계 정보로부터 개개에 대한 정보를 추적하지 못하도록 해야 함

* 데이터베이스 보안 통제(제어)
- 접근 통제 : 데이터 베이스 사용자가 가진 접근 권한의 범위 내에서 데이터 접근을 허용하는 기술적인 방법 -> DBA 는 접근 통제를 위해 개별 사용자 또는 역할/그룹별로 접근 가능한 데이터, 데이터의 접근 수준, 데이터에 허용할 작업 등을 결정한다. -> 접근 통제를 위한 체계적인 정책을 확립하고 이를 수행 해야 함

- 추론 통제 : 통계 정보 등 간접적으로 노출된 데이터를 통해서 민감/기밀 데이터가 유추되어 공개되는 것을 방지하는 것을 말함. -> 완벽한 해결책이 존재하지는 않지만 허용 가능한 질의를 제한하고 질의의 응답으로 제공되는 데이터를 한정한다. 데이터가 숫자인 경우 반올림하거나 일관성이 없는 질의 결과를 제공한다

- 흐름 통제(제어) : 접근이 가능한 객체 간의 정보의 흐름(A 테이블의 데이터가 B에 기록되었다면 A에서 B로 흐름이 발생했다고 표현함)을 조정하는것을 말함 -> 보안 등급이 높은 객체에서 보안 등급이 낮은 객체로의 정보 흐름을 제어해야 함

* DBMS 보안 통제 : DML(Select, Update, delete, Insert), DDL(Create, Drop, Alter), DCL(Grant, Revoke), TCL (Commit, Rollback)

* 뷰 기반의 접근 통제 : 하나 이상의 기본 테이블로 부터 유도되어 만들어지는 가상의 테이블로 일반적으로 뷰의 내용은 실제로 존재하는 것이 아니라 뷰에 대한 조작 유구시마다 기본 테이블의 데이터를 이용해서 만들어진다 -> 실행시간에 구체화 되는 특성을 가짐
-> 뷰의 정의로 권한 부여 기법을 사용할 수 있음 -> 기본 테이블에 직접 접근 차단, 기본 테이블에서 허용하는 일부 컬럼에 대해서만 뷰를 통해 제공 -> 권한 없는 사용자로부터 민감 데이터 은닉 가능

* SQL 기반의 접근 통제 : Grant, revoke 같은 DCL 문을 사용하면 데이터 베이스 객체에 대한 작업 권한을 제한할 수 있음

-> GRANT [부여할 권한] ON [데이터베이스 객체 이름] TO [사용자] [WITH GRANT OPTION]
-> with grant option 을 사용하면 사용자가 부여받은 권한을 다른 사용자에게 또 다시 부여할 수 있음
-> REVOKE [해제할 권한] ON [데이터베이스 객체] FROM [사용자]
-> show grants for [사용자] // 부여한 권한 확인

* 데이터베이스 암호화 기술
- 컬럼 암호화 방식 : 테이블의 컬럼 단위로 데이터를 암호화 하여 저장하는 방식 -> API방식(암복화 모듈이 API 라이브러리 형태로 애플리케이션 서버에 설치되어 응용 프로그램에서 암/복호화 모듈을 호출하는 방식, 애플리케이션 서버와 DB 서버 간의 통신에서 암호화된 데이터 전송을 보장할 수 있음, 애플리케이션이 변경되긴 해야 해서 추가 개발 비용이 발생함), Plug-in방식 (DB 서버에서 플러그인으로 연결된 암/복호화 모듈을 호출하는 방식, 암복호화 모듈은 DB서버에 저장 됨, 테이블/컬럼 암호화시 기존 테이블 이름과 동일한 뷰와 변경된 이름의 암호화된 테이블이 생성 되고 원본 테이블은 삭제 됨, DB 서버에서 암복호화 모듈을 플러그인으로 호출할 때 추가적인 부하가 발생하여 성능이 저하될 수 있으므로 트랜잭션 처리량이 많지 않아 성능에 대한 민감성이 낮은 시스템에 적용하기 적합함), Hybrid방식 (Plugin 방식을 사용해서 성능저하가 발생할 수 있는 부분은 API 방식을 사용하여 최적의 성능을 보장하고 나머지 부분들은 Plugin 방식을 사용하여 애플리케이션의 수정을 최소하는 방식)

=> 암복호화 모듈이 설치되는 위치가 애플리케이션이면 API, DB서버면 Plug-in 방식이다

- 블록 암호화 방식 : DBMS 블록 또는 파일 블록 단위로 암호화하여 저장하는 방식 -> TDE방식(Transparent Data Encryption 방식, DB자체 암호화 방식을 의미함. DBMS 에 내장된 암호화 기능을 이용하여 DB 내부에서 데이터 파일 저장시 암호화 하고 파일에 저장된 내용을 메모리 영역으로 가져올 때 DBMS에 의해 자동으로 복호화 하는 방식, DBMS 커널 수준에서 처리되므로 기존 응용 프로그램의 수정이나 DB 스키마의 변경이 거의 필요하지 않고 DBMS 엔진에 최적화된 성능을 제공할 수 있음), 파일 암호화 방식 (운영체제 암호화 방식을 의미함, 운영체제에서 파일을 암호화는 방식으로 파일에 대해 직접 암호화를 수행하므로 DB파일 뿐만 아니라 로그 파일, 이미지 파일, 음성/영상 등의 다양한 비정형 데이터에 대한 암호화 적용이 필요하다 -> DBMS 종류에 상관없이 사용할 수 있는 장점이 있지만 DB 서버에 추가적인 부하가 발생할 수 있음)

* mysql DBMS 취약점 점검
- 디폴트 관리자 패스워드 및 계정명을 변경한다 (Brute Force/Dictionary 공격 등으로 추측하기 어려운 이름으로 변경)
- mysql 설치 시 서버에 자동으로 생성되는 mysql 계정의 로그인을 차단함 -> /bin/false, /sbin/nologin 으로 변경하여 로그인할 수 없도록 함
- 원격에서 mysql 서버로의 접속을 차단함 -> 3306/tcp 포트를 차단하여 로컬에 설치된 애플리케이션에 의해서만 접근할 수 있도록 함 , my.cnf 파일에 skip-networking 를 추가하면 네트워킹 기능을 비활성화 하기 때문에 서비스 포트를 오픈하지 않음 -> 로컬에서는 통신 가능함 (로컬에서는 mysql.sock 으로 통신함)
- 데이터베이스의 사용자별 접속/권한 확인 : 일반 사용자에게 File, Proccess, Shutdown 권한이 주어질 경우 해커의 시스템 침입 대상이 될 수 있음 (File_priv -> 파일에 데이터를 쓰고 읽을 수 있는 권한 , Process_priv -> show processlist 명령 실행을 통해 실행중인 질의를 모니터링 할 수 있게 됨 , Shutdown_priv -> mysqladmin shutdown 명령어를 이용하여 mysql 서버를 종료시킬 수 있는 권한)
- mysql 버전 확인 및 보안 패키 적용 여부 점검
- mysql 데이터 디렉터리 보호 여부 점검 (mysql 은 테이블의 데이터 및 로그를 파일 형태로 관리함. 이 파일은 데이터 디렉터리에 저장되므로 디렉터리 권한을 잘못 설정할 경우 서버의 일반 사용자 또는 서버를 해킹한 해커에 의해 데이터 및 로그가 삭제되는 사고가 발생할 수 있음)

* 클라우드 컴퓨팅 보안 : 집적/공유된 정보통신기기, 정보통신설비, 소프트웨어 등 정보통신 작원을 이용자의 요구나 수요 변화에 따라 정보통신망을 통하여 유동적으로 이용할 수 있도록 하는 정보처리 체계를 의미함 -> 인터넷을 통해서 이용자가 IT 자원을 필요한만큼 빌려서 사용하고 서비스 부하에 따라 실시간 확장성을 지원 받으며 사용한 만큼 비용을 지불함

* 가상화 : 클라우드 컴퓨팅의 핵심기술로 물리적인 IT 자원을 동시에 다수의 논리적인 IT자원으로 사용할 수 있는 기술 -> 데스크톱 가상화, 애플리케이션 가상화, 스토리지 가상화, 서버 가상화, 네트워크 가상화 등이 있음

* 서비스 모델(Delivery Model) -> 이용자가 클라우드 컴퓨팅 서비스에 접근할 수 있는 형태에 따라 분류
- IaaS(Infrastructure as a Service) : 하드웨어 인프라 자원만을 사용량 기반으로 제공하는 서비스, 인프라 자원을 가상화 기술을 통해 가상화된 환경에서 쉽고 신속하게 할당 받아 사용 가능 -> AWS

- Paas(Platform as a Service) -> 이용자에게 애플리케이션을 개발, 테스트, 실행하는데 필요한 운영체제, 개발 환경 플랫폼(WAS,WEB,DB), 개발 언어 프레임워크(스프링, Node.js) 등을 제공하는 서비스 -> MS Azure, Google AppEngine, Oracle Paas Platform

- Saas(Software as as Service) -> 이용자들에게 클라우드 환경에서 동작하는 애플리케이션/소프트웨어를 제공하는 서비스 -> dropbox, 네이버 N 클라우드, 구글 드라이브

* 배치모델(Deployment Model) ->구현을 어떻게 하는가에 따라 분류
- 퍼블릭 클라우드 모델 : 불특정 다수를 대상으로 하는 서비스 -> 여러 서비스 사용자가 이용하는 형태 (커스터마이징이 힘듬)
- 프라이빗 클라우드 모델 : 기업 및 기관 내부에 클라우드 솔루션을 이용하여 클라우드 서비스 환경을 구축하고 내부자에게 제한적으로 서비스를 제공하는 형태를 말함 (비쌈, 시간이 오래걸림)
- 하이브리드 클라우드 모델 : 퍼블릭 클라우드와 프라이빗 클라우드가 결합된 형태로 공유를 원하지 않는 일부 데이터 및 서비스에 대해 프라이빗 정책을 설정하여 서비스를 제공함

* SecaaS (Security as a Service) -> 클라우드 기반 보안 서비스, Saas 의 한 종류로 클라우드 기반 보안 또는 보안 클라우드라고 명명함. 클라우드를 통해 주문형 방식으로 제공하는 보안 서비스 모델 -> 필요한 만큼의 보안 서비스를 이용하고 요금을 지불함

* 클라우드 보안 연합 : CSA (Cloud Security Alliance) 정의
-> 식별/접근 관리 : IAM (Identify and Access Management), 인증/신원보증/정보 접근권한이 있는 사용자 관리를 포함한 관리 및 접근 제어 제공
-> 데이터 유출, 손실 방지 = DLP(Data Loss Prevention), 이동 및 사용중인 데이터의 보안 및 모니터링, 보안 검증
-> 웹 보안 : 클라우드 서비스 공급자를 통해 웹 트리팩을 프록시 처리하여 일반적으로 제공되는 공개된 애플리케이션 서비스를 실시간 보호함
-> 이메일 보안 : 인바운드 및 아웃바운드 Email을 제어하고 피싱 및 악의적인 첨부파일과 스팸으로부터 조직을 보호함
-> 보안 감사 : 업계 표준을 기반으로 클라우드 서비스에 대한 제 3자 감사
-> 침입 관리 : 패턴 인식을 통하여 통계적으로 이벤트를 감지, 침입 시도 방지 또는 탐지를 통해 관리
-> 보안 정보 및 이벤트 관리 = SIEM, Security Information & Event Management : 보안 로그 및 이벤트 정보에 대한 상관관계 및 실시간 분석
-> 암호화 : 해독할 수 없도록 데이터를 암호화
-> 업무 연속성과 재난 복구 = BCDR, Business Continuity and Disaster Recovery : 서비스 중단 시 운영의 탄력성을 보장하도록 설계된 조치
-> 취약점 검사 : 공용 네트워크를 통해 대상 인프라 또는 시스템의 취약점을 검사
-> 지속적인 모니터링 : 조직의 현재 보안 상태를 나타내는 지속적인 위험 관리 수행